



Facultad de
**Ciencias Sociales y
Tecnologías de la Inf.**
Talavera de la Reina. UCLM

UNIVERSIDAD DE CASTILLA-LA MANCHA
Tecnologías y Sistemas de Información

GRADO EN INGENIERÍA INFORMÁTICA
SISTEMAS DE INFORMACIÓN

Trabajo Fin de Grado

**Creación de una Plataforma/API para la
Evaluación de Calidad de Datos
en Proyectos de IA**

Autoría: Alejandro Manuel Saiz García

Tutoría: Fernando Gualo Cejudo

Cotutoría: Ricardo Pérez del Castillo

Junio, 2026

Alejandro Manuel Saiz García

Talavera de la Reina – España

© 2026 Alejandro Manuel Saiz García

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Se permite la copia, distribución y/o modificación de este documento bajo los términos de la Licencia de Documentación Libre GNU, versión 1.3 o cualquier versión posterior publicada por la *Free Software Foundation*; sin secciones invariantes. Una copia de esta licencia esta incluida en el apéndice titulado «GNU Free Documentation License».

Muchos de los nombres usados por las compañías para diferenciar sus productos y servicios son reclamados como marcas registradas. Allí donde estos nombres aparezcan en este documento, y cuando la persona autora haya sido informada de esas marcas registradas, los nombres estarán escritos en mayúsculas o como nombres propios.

0.

TRIBUNAL:

Presidencia:

Vocal:

Secretaría:

FECHA DE DEFENSA:

CALIFICACIÓN:

PRESIDENCIA

VOCAL

SECRETARÍA

Fdo.:

Fdo.:

Fdo.:

Resumen

La calidad de los datos constituye un factor determinante en el éxito de los proyectos de inteligencia artificial y aprendizaje automático. Sin embargo, las herramientas disponibles para su evaluación presentan limitaciones significativas: las bibliotecas basadas en código imponen una barrera técnica elevada, mientras que los *scripts* ad hoc carecen de estandarización, historización y mecanismos de decisión automatizados.

Este Trabajo Fin de Grado presenta DATAQUAL, una plataforma web *full-stack* para la evaluación automatizada de la calidad de datos. La plataforma implementa siete métricas de calidad alineadas con los estándares ISO/IEC 25012 e ISO/IEC 5259, un motor extensible basado en *plugins*, un sistema de *fingerprinting* SHA-256 para la trazabilidad de incidencias entre evaluaciones y *Quality Gates* configurables inspirados en SonarQube. El sistema se despliega como ocho servicios Docker orquestados con Docker Compose, combinando una interfaz web interactiva (Next.js, React) con una API RESTful (Flask, Python) y procesamiento asíncrono (Celery, Redis).

La plataforma ha sido validada con conjuntos de datos reales, demostrando su capacidad para diagnosticar problemas de calidad, rastrear su evolución y proporcionar señales claras de aptitud de los datos mediante *Quality Gates* con estados PASSED, WARNING y FAILED.

Abstract

Data quality is a determining factor in the success of artificial intelligence and machine learning projects. However, the tools available for its assessment have significant limitations: code-based libraries impose a high technical barrier, while ad hoc scripts lack standardisation, historisation and automated decision mechanisms.

This undergraduate thesis presents DATAQUAL, a full-stack web platform for automated data quality evaluation. The platform implements seven quality metrics aligned with the ISO/IEC 25012 and ISO/IEC 5259 standards, an extensible plugin-based engine, a SHA-256 fingerprinting system for issue traceability across evaluations, and configurable Quality Gates inspired by SonarQube. The system is deployed as eight Docker services orchestrated with Docker Compose, combining an interactive web interface (Next.js, React) with a RESTful API (Flask, Python) and asynchronous processing (Celery, Redis).

The platform has been validated with real-world datasets, demonstrating its ability to diagnose quality problems, track their evolution, and provide clear data fitness signals through Quality Gates with PASSED, WARNING and FAILED states.

A mis padres

Índice general

Resumen	III
Abstract	IV
Índice general	VI
Índice de cuadros	XIII
Índice de figuras	XV
Listado de acrónimos	XVI
1. Introducción	1
1.1. Motivación	1
1.1.1. Impacto en la fiabilidad de los modelos	1
1.1.2. Coste económico	2
1.1.3. Tiempo dedicado a la preparación de datos	2
1.1.4. Sesgo algorítmico	2
1.1.5. Requisitos normativos	2
1.2. Presentación de la solución	3
1.3. Por qué una plataforma dedicada	3
1.4. Público objetivo	4
1.5. Alcance del proyecto	5
1.6. Estructura del documento	5
2. Antecedentes y estado del arte	7
2.1. La importancia de la calidad de datos en proyectos de IA	7

2.1.1.	Marcos conceptuales fundacionales	7
2.1.2.	Impacto económico de la mala calidad de datos	7
2.1.3.	El paradigma data-centric AI	8
2.2.	Marco normativo: estándares ISO	9
2.2.1.	ISO/IEC 25012: Modelo de calidad de datos	9
2.2.2.	ISO/IEC 5259: Calidad de datos para analítica e IA	9
2.2.3.	ISO/IEC 25024: Medición de la calidad de datos	10
2.3.	Estado del arte: herramientas existentes	11
2.3.1.	Great Expectations	11
2.3.2.	Apache Deequ	11
2.3.3.	DQOps	12
2.3.4.	Herramientas comerciales	12
2.3.5.	Análisis comparativo	12
2.4.	Diferenciación de DATAQUAL	13
2.5.	Justificación académica	14
2.5.1.	Ingeniería del Software	14
2.5.2.	Sistemas distribuidos	14
2.5.3.	Bases de datos	14
2.5.4.	Desarrollo web	15
2.5.5.	Análisis de datos y estadística	15
2.5.6.	Seguridad informática	15
2.5.7.	DevOps y despliegue	15
3.	Objetivos	16
3.1.	Objetivo general	16
3.2.	Objetivos específicos	16
3.2.1.	OE1. Módulo de ingesta y almacenamiento	17
3.2.2.	OE2. Motor de evaluación con métricas parametrizables	17
3.2.3.	OE3. API RESTful documentada	18
3.2.4.	OE4. Aplicación web con <i>dashboards</i> interactivos	19
3.2.5.	OE5. Mecanismos de identificación de problemas	20

0. ÍNDICE GENERAL

3.3. Objetivos transversales	20
3.4. Alcance y limitaciones	21
3.4.1. Dentro del alcance	21
3.4.2. Fuera del alcance	21
3.4.3. Restricciones tecnológicas	22
4. Metodología y herramientas	23
4.1. Gestión del proyecto	23
4.1.1. Metodología ágil adoptada: Scrum adaptado	23
4.1.2. Adaptación de Scrum al TFG individual	23
4.1.3. Herramientas de gestión	24
4.2. Metodología de desarrollo	24
4.2.1. Desarrollo iterativo	25
4.2.2. Prototipado	25
4.2.3. Control de versiones	25
4.3. Pila tecnológica	25
4.3.1. <i>Frontend</i>	26
4.3.2. <i>Backend</i>	26
4.3.3. Base de datos y almacenamiento	26
4.3.4. Procesamiento asíncrono y orquestación	26
4.4. Entorno de desarrollo	26
4.5. Estrategia de pruebas	28
4.5.1. Pruebas automatizadas	28
4.5.2. Pruebas manuales	29
4.6. Planificación temporal	29
4.6.1. Fase 1: Análisis y diseño inicial (octubre–noviembre 2025)	29
4.6.2. Fase 2: Implementación del núcleo (diciembre 2025–enero 2026)	29
4.6.3. Fase 3: Motor de métricas (febrero 2026)	29
4.6.4. Fase 4: Interfaz de usuario y visualización (marzo 2026)	29
4.6.5. Fase 5: Funcionalidades avanzadas (abril 2026)	29
4.6.6. Fase 6: Documentación y defensa (mayo–junio 2026)	29

4.6.7. Diagrama de Gantt	30
4.7. Backlog inicial e historias de usuario	30
4.8. Estimación de esfuerzo	30
4.8.1. Técnica de estimación	30
4.8.2. Estimación inicial vs. real	30
4.9. Estimación de costes	32
4.10. Gestión de riesgos	33
5. Resultados: desarrollo iterativo por sprints	35
5.1. Requisitos globales del sistema	35
5.1.1. Historias de usuario	35
5.1.2. Casos de uso principales	35
5.1.3. Burndown global del proyecto	35
5.2. Sprint 1 — Análisis, arquitectura y setup inicial	36
5.2.1. Planificación del sprint	36
5.2.2. Trabajo realizado	37
5.2.3. Cómo se ha hecho	37
5.2.4. Retrospectiva	39
5.3. Sprint 2 — <i>Backend</i> core: datasets, API REST	40
5.3.1. Planificación del sprint	40
5.3.2. Trabajo realizado	40
5.3.3. Cómo se ha hecho	41
5.3.4. Retrospectiva	42
5.4. Sprint 3 — Motor de métricas de calidad	43
5.4.1. Planificación del sprint	43
5.4.2. Trabajo realizado	43
5.4.3. Cómo se ha hecho	43
5.4.4. Retrospectiva	46
5.5. Sprint 4 — <i>Frontend</i> e interfaz de usuario	47
5.5.1. Planificación del sprint	47
5.5.2. Trabajo realizado	47

0. ÍNDICE GENERAL

5.5.3. Cómo se ha hecho	47
5.5.4. Retrospectiva	50
5.6. Sprint 5 — Funcionalidades avanzadas	50
5.6.1. Planificación del sprint	50
5.6.2. Trabajo realizado	51
5.6.3. Cómo se ha hecho	51
5.6.4. Retrospectiva	52
5.7. Sprint 6 — Pruebas, refinamiento y documentación	53
5.7.1. Planificación del sprint	53
5.7.2. Trabajo realizado	54
5.7.3. Cómo se ha hecho	54
5.7.4. Retrospectiva	55
6. Resultados y discusión	57
6.1. Resultados alcanzados	57
6.1.1. Resultados cuantitativos	57
6.1.2. Resultados cualitativos	57
6.1.3. Verificación del cumplimiento de objetivos	58
6.2. Fortalezas de la plataforma	58
6.3. Limitaciones	59
6.4. Comparación con herramientas existentes	60
7. Conclusiones y trabajo futuro	61
7.1. Revisión de objetivos	61
7.2. Conclusiones	62
7.3. Lecciones aprendidas	63
7.4. Líneas de trabajo futuro	63
7.5. Aportación del proyecto	64
7.5.1. Aportación técnica	64
7.5.2. Aportación académica	64
7.5.3. Aportación profesional	65

7.6. Competencias adquiridas	65
7.6.1. Competencias técnicas	65
7.6.2. Competencias metodológicas	65
7.6.3. Competencias transversales	66
7.7. Valoración personal	66
A. Especificación de la API RESTful	68
A.1. Módulo de autenticación (/api/auth)	68
A.2. Módulo de proyectos (/api/projects)	68
A.3. Módulo de conjuntos de datos (/api/datasets)	69
A.4. Módulo de métricas (/api/metrics)	69
A.5. Módulo de evaluaciones (/api/evaluations)	69
A.6. Módulo de administración (/api/admin)	70
B. Catálogo de métricas de calidad	71
B.1. Completitud (completeness)	71
B.2. Unicidad (uniqueness)	71
B.3. Valores atípicos (outliers)	72
B.4. Exactitud sintáctica (syntactic_accuracy)	72
B.5. Consistencia lógica (logical_consistency)	73
B.6. Equilibrio de clases (class_balance)	73
B.7. Actualidad (currentness)	74
C. Backlog completo del proyecto	75
D. Plan de riesgos detallado	77
E. Estimación de costes y viabilidad económica	79
F. Manual de usuario	81
G. Diagramas UML del sistema	84
Referencias	87

Índice de cuadros

2.1. Correspondencia entre características de ISO/IEC 25012 y métricas de DATAQUAL	10
2.2. Comparativa de DATAQUAL con herramientas de calidad de datos existentes	13
4.1. Tecnologías del <i>frontend</i>	26
4.2. Tecnologías del <i>backend</i>	27
4.3. Tecnologías de base de datos y almacenamiento	27
4.4. Tecnologías de procesamiento asíncrono y orquestación	28
4.5. Diagrama de Gantt del proyecto (curso 2025–2026)	30
4.6. Backlog inicial — historias de usuario principales	31
4.7. Estimación inicial vs. real por sprint	31
4.8. Estimación de costes del proyecto	32
4.9. Plan de gestión de riesgos	34
5.1. Sprint 1 — Historias de usuario y estimación	36
5.2. Servicios Docker de DATAQUAL	37
5.3. Protocolos de comunicación entre componentes	38
5.4. Sprint 2 — Historias de usuario y estimación	40
5.5. Sprint 3 — Historias de usuario y estimación	43
5.6. Sprint 4 — Historias de usuario y estimación	47
5.7. Sprint 5 — Historias de usuario y estimación	50
5.8. Sprint 6 — Actividades y estimación	53
6.1. Resultados cuantitativos de la implementación	57
6.2. Verificación del cumplimiento de los objetivos específicos	58

6.3. Comparación de DATAQUAL con herramientas existentes (perspectiva post-implementación)	60
C.1. Backlog Sprint 1	75
C.2. Backlog Sprint 2	75
C.3. Backlog Sprint 3	76
C.4. Backlog Sprint 4	76
C.5. Backlog Sprint 5	76
C.6. Backlog Sprint 6	76
D.1. Registro de riesgos con ocurrencia y acciones aplicadas	78
E.1. Horas reales de desarrollo por sprint y categoría	79
E.2. Desglose completo de costes del proyecto	80
E.3. Estimación de costes de producción en AWS (por mes)	80

Índice de figuras

5.1. Burndown global del proyecto: <i>story points</i> restantes por sprint (línea ideal vs. progreso real)	36
5.2. Cuadro de mando principal de DATAQUAL: tarjetas resumen de proyectos, distribución de estados de <i>Quality Gate</i> y alertas de proyectos con problemas detectados	48
5.3. Página de resultados de evaluación: indicador circular de puntuación de calidad (<i>gauge</i>), tabla de métricas por columna y listado de <i>issues</i> con severidad y columna afectada	49
5.4. Módulo de exploración de datos (EDA): histogramas de distribución, diagramas de caja y matriz de correlación Pearson/Spearman para columnas numéricas	49
5.5. <i>Quality Gate</i> de DATAQUAL: estado global con código de color (PASSED/WARNING/FAILED), desglose de condiciones configuradas y <i>badges</i> de trazabilidad de <i>issues</i> por <i>fingerprinting</i>	52

Listado de acrónimos

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DMBOK	Data Management Body of Knowledge
EDA	Exploratory Data Analysis
ETL	Extract, Transform, Load
IA	Inteligencia Artificial
IQR	Interquartile Range
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
MLOps	Machine Learning Operations
MVC	Model-View-Controller
ORM	Object-Relational Mapping
PBKDF2	Password-Based Key Derivation Function 2
PII	Personally Identifiable Information
REST	Representational State Transfer
RGPD	Reglamento General de Protección de Datos
SAAS	Software as a Service
SHA	Secure Hash Algorithm
SQL	Structured Query Language
SQUARE	Systems and Software Quality Requirements and Evaluation
TFG	Trabajo Fin de Grado
UI	User Interface

0. LISTADO DE ACRÓNIMOS

WSGI	Web Server Gateway Interface
XLSX	Microsoft Excel Open XML Spreadsheet

Capítulo 1

Introducción

LA calidad de los datos constituye uno de los pilares fundamentales sobre los que se sustenta el éxito de cualquier proyecto de Inteligencia Artificial (IA) o Machine Learning (ML). A medida que las organizaciones adoptan sistemas basados en datos para la toma de decisiones estratégicas, la necesidad de garantizar que dichos datos sean completos, precisos, consistentes y oportunos se ha convertido en un requisito ineludible. Sin embargo, la realidad de la industria muestra que la calidad de los datos sigue siendo un problema generalizado y costoso, tanto en términos económicos como en sus consecuencias sobre la fiabilidad de los modelos resultantes. El presente Trabajo Fin de Grado (TFG) aborda esta problemática mediante el diseño e implementación de DATAQUAL, una plataforma web integral para la evaluación automatizada de la calidad de datos en proyectos de IA.

Este capítulo introductorio ofrece una visión general del problema que motiva el desarrollo de la plataforma, presenta la solución propuesta y sus principales capacidades, justifica la necesidad de una herramienta dedicada frente a los enfoques tradicionales, identifica el público objetivo al que se dirige y describe la estructura del resto del documento.

1.1 Motivación

El principio conocido como *garbage in, garbage out* resume de forma elocuente una verdad ampliamente reconocida en el ámbito de la ciencia de datos: la calidad de las salidas de un sistema está intrínsecamente limitada por la calidad de sus entradas [WS96]. En el contexto de la IA y el ML, este principio adquiere una relevancia especial, ya que los modelos aprenden patrones directamente de los datos de entrenamiento. Si estos datos contienen errores, sesgos o valores ausentes, los modelos resultantes heredarán —y, con frecuencia, amplificarán— dichas deficiencias [Red98].

1.1.1 Impacto en la fiabilidad de los modelos

Diversos estudios han demostrado que los problemas de calidad de datos constituyen la causa principal de fallos de sistemas de IA en producción, por encima incluso de los errores algorítmicos. Sambasivan et al. [SKH⁺21] documentaron, a partir de entrevistas con 53 profesionales de IA en múltiples organizaciones, que las deficiencias en los datos —etiquetado incorrecto, datos desactualizados, muestras no representativas— son la causa predominante

1. INTRODUCCIÓN

de los fracasos en despliegues de ML. A modo ilustrativo, se ha estimado que un 10 % de etiquetas incorrectas en un conjunto de datos de entrenamiento puede reducir la precisión de un clasificador en más de 15 puntos porcentuales, una degradación que ninguna mejora arquitectónica puede compensar de forma fiable [Ng21].

1.1.2 Coste económico

El impacto económico de la mala calidad de datos es igualmente significativo. Según estimaciones de Gartner, las organizaciones pierden una media de 12,9 millones de dólares anuales como consecuencia directa de datos deficientes [Gar17]. En el ámbito de la IA, este coste se amplifica de manera notable: modelos entrenados con datos erróneos producen decisiones incorrectas que se traducen en consecuencias financieras, legales y reputacionales. Redman [Red01] señaló que el coste de corregir errores en los datos crece exponencialmente a medida que estos avanzan por las etapas de un flujo de procesamiento, lo que subraya la importancia de detectarlos lo antes posible.

1.1.3 Tiempo dedicado a la preparación de datos

Encuestas realizadas por plataformas de referencia como Kaggle [Kag22] revelan que entre el 60 % y el 80 % del tiempo de los profesionales de datos se dedica a tareas de limpieza y preparación, en detrimento del tiempo disponible para el análisis, la experimentación y el desarrollo de modelos. Este desequilibrio evidencia la necesidad de herramientas que automaticen y sistematicen los procesos de evaluación de calidad, permitiendo a los equipos concentrar sus esfuerzos en actividades de mayor valor añadido.

1.1.4 Sesgo algorítmico

Los datos desequilibrados o no representativos constituyen una fuente frecuente de sesgo en los modelos de IA. El caso del algoritmo de selección de personal de Amazon, que fue descartado en 2018 tras detectarse que discriminaba sistemáticamente a las candidatas femeninas [Das18], puso de manifiesto los riesgos de entrenar modelos con datos históricos sesgados. De forma análoga, Buolamwini y Gebru [BG18] demostraron que los sistemas comerciales de reconocimiento facial presentaban tasas de error significativamente superiores para personas de piel oscura, especialmente mujeres, como consecuencia de conjuntos de entrenamiento insuficientemente diversos. Estos casos ilustran que la calidad de los datos no es únicamente una cuestión técnica, sino también ética y social.

1.1.5 Requisitos normativos

El marco regulatorio refuerza la necesidad de garantizar la calidad de los datos. El Reglamento Europeo de IA (*AI Act*) [Par24] establece requisitos explícitos de calidad, trazabilidad y documentación de los datos utilizados en sistemas de IA de alto riesgo. De manera complementaria, el Reglamento General de Protección de Datos (RGPD) [Par16] exige que los

datos personales sean exactos y estén actualizados, imponiendo obligaciones de rectificación y eliminación que afectan directamente a los flujos de datos de proyectos de ML. En este contexto normativo, disponer de mecanismos formales y auditables de evaluación de la calidad de datos deja de ser una buena práctica para convertirse en una obligación legal.

1.2 Presentación de la solución

Para abordar la problemática descrita, se ha diseñado e implementado DATAQUAL, una plataforma web *full-stack* para la evaluación automatizada de la calidad de datos en proyectos de IA, ML y minería de datos. La plataforma combina una interfaz gráfica interactiva con una Application Programming Interface (API) Representational State Transfer (REST)ful [Fie00], ofreciendo las siguientes capacidades principales:

- **Carga de conjuntos de datos** en múltiples formatos (Comma-Separated Values (CSV), Microsoft Excel Open XML Spreadsheet (XLSX), JavaScript Object Notation (JSON) y Parquet), con almacenamiento seguro y versionado.
- **Configuración de métricas de calidad** adaptadas al tipo de proyecto, con parámetros y umbrales personalizables alineados con los estándares International Organization for Standardization (ISO)/IEC 25012 [ISO08] y ISO/IEC 5259 [ISO24].
- **Ejecución asíncrona de evaluaciones**, que analiza los conjuntos de datos frente a las métricas configuradas sin bloquear la interfaz de usuario.
- **Visualización interactiva de resultados** mediante cuadros de mando que incluyen gráficos de evolución, tablas de métricas por columna, histogramas, diagramas de caja y matrices de correlación.
- **Seguimiento de la evolución de la calidad** a lo largo del tiempo, gracias a un sistema de *fingerprinting* que permite comparar automáticamente los resultados con líneas base anteriores.
- **Quality Gates** (puertas de calidad): umbrales mínimos configurables que determinan si un conjunto de datos es apto para su uso, con estados PASSED, WARNING y FAILED.
- **Protección de datos sensibles** mediante la detección y enmascaramiento automático de información personal identificable (Personally Identifiable Information (PII)).
- **Despliegue contenerizado** como ocho servicios Docker orquestados con Docker Compose, garantizando portabilidad, reproducibilidad y soberanía sobre los datos.

1.3 Por qué una plataforma dedicada

Los enfoques tradicionales para la evaluación de la calidad de datos —*scripts* ad hoc en Python, consultas Structured Query Language (SQL) manuales o cuadernos Jupyter —presentan limitaciones fundamentales que dificultan su adopción sistemática en entornos profesionales:

1. INTRODUCCIÓN

- **Falta de estandarización.** Cada equipo define sus propias métricas y umbrales de forma aislada, lo que impide la comparación entre proyectos y dificulta la auditoría. DATAQUAL implementa un catálogo de métricas alineado con ISO/IEC 25012 [ISO08] y la serie ISO/IEC 5259 [ISO24], proporcionando un marco de referencia común basado en el modelo Systems and Software Quality Requirements and Evaluation (SQUARE).
- **Ausencia de historización.** Los *scripts* ad hoc evalúan instantáneas puntuales sin mantener un registro histórico. DATAQUAL conserva el historial completo de ejecuciones y permite la comparación automática con líneas base anteriores, facilitando la detección de degradaciones en la calidad.
- **Barrera técnica.** Las herramientas basadas en código, como Great Expectations [Gre24] o Deequ de Amazon, requieren conocimientos de programación que no todos los perfiles involucrados en un proyecto de datos poseen. DATAQUAL proporciona una interfaz web accesible que permite a usuarios no técnicos configurar y ejecutar evaluaciones de calidad sin necesidad de escribir código.
- **Falta de integración en flujos de trabajo.** Los *scripts* independientes no se integran de forma natural en *pipelines* de datos existentes. La API REST de DATAQUAL permite la integración programática con flujos de Machine Learning Operations (MLOPS), integración y despliegue continuos, y procesos Extract, Transform, Load (ETL).
- **Ausencia de Quality Gates.** Pocas herramientas proporcionan señales claras de aprobación o rechazo sobre la calidad de un conjunto de datos. Inspirándose en el concepto de *Quality Gate* de SonarQube [Son24] para la calidad del código, DATAQUAL implementa puertas de calidad configurables con estados PASSED, WARNING y FAILED, ofreciendo criterios objetivos y automatizados para la toma de decisiones.

1.4 Público objetivo

DATAQUAL ha sido diseñada para ser utilizada por los distintos perfiles profesionales que intervienen en el ciclo de vida de un proyecto de datos:

- **Ingenieros de datos**, responsables de la ingestión, transformación y almacenamiento de los conjuntos de datos, que pueden integrar las evaluaciones de calidad en sus *pipelines* ETL mediante la API REST.
- **Científicos de datos e ingenieros de ML**, que necesitan verificar la calidad de los datos de entrenamiento y evaluación antes de construir y desplegar modelos.
- **Analistas de datos**, que requieren una herramienta visual e intuitiva para explorar la calidad de los conjuntos de datos sin depender de conocimientos de programación.
- **Responsables de calidad y cumplimiento normativo**, que necesitan evidencias auditables del estado de la calidad de los datos para satisfacer los requisitos del RGPD y el Reglamento Europeo de IA.

- **Directores técnicos de proyecto**, que precisan una visión consolidada del estado de la calidad de los datos a través de cuadros de mando y métricas agregadas para la toma de decisiones informadas.

1.5 Alcance del proyecto

DATAQUAL es una plataforma *full-stack* orientada a proyectos de IA y ML de escala mediana (conjuntos de datos de hasta 100 MB). El alcance del proyecto incluye:

- Gestión de proyectos y conjuntos de datos con versionado y almacenamiento seguro en MinIO.
- Motor de evaluación con seis métricas de calidad evaluables alineadas con ISO/IEC 25012, más detección de valores atípicos disponible en el módulo de perfilado de datos.
- API REST documentada con autenticación JSON Web Token (JWT).
- Interfaz web interactiva con cuadros de mando, perfilado exploratorio de datos y Quality Gates.
- Sistema de *fingerprinting* Secure Hash Algorithm (SHA)-256 para el seguimiento determinista de problemas entre evaluaciones.
- Protección nativa de datos PII con enmascaramiento en todas las métricas.
- Despliegue contenerizado mediante Docker Compose.

Quedan **fuera del alcance** del presente trabajo: el procesamiento de conjuntos de datos superiores a 100 MB, el análisis de flujos en tiempo real (*streaming*), la corrección automática de errores, la integración nativa con plataformas *cloud* propietarias (AWS, Azure, GCP) y la internacionalización de la interfaz.

1.6 Estructura del documento

El presente documento se organiza en siete capítulos y siete anexos. A continuación se describe brevemente el contenido de cada uno de los capítulos restantes:

Capítulo 2: Antecedentes y estado del arte

Revisa los fundamentos teóricos de la calidad de datos, los principales estándares internacionales (ISO/IEC 25012, ISO/IEC 5259), las dimensiones de calidad relevantes y las herramientas existentes en el mercado, identificando las carencias que justifican el desarrollo de DATAQUAL.

Capítulo 3: Objetivos

Define el objetivo general y los objetivos específicos del proyecto, estableciendo el alcance detallado y las restricciones del trabajo.

1. INTRODUCCIÓN

Capítulo 4: Metodología y herramientas

Describe la metodología de gestión y desarrollo adoptada, la pila tecnológica, el backlog inicial, la planificación (Gantt), la estimación de esfuerzo, los costes y la gestión de riesgos.

Capítulo 5: Resultados: desarrollo iterativo por sprints

Desarrolla el proceso de implementación iterativo por sprints, incluyendo para cada sprint la planificación, el trabajo realizado, las decisiones técnicas adoptadas y la retrospectiva.

Capítulo 6: Resultados y discusión

Presenta los resultados globales obtenidos, analiza el cumplimiento de los objetivos planteados y compara DATAQUAL con las herramientas existentes.

Capítulo 7: Conclusiones y trabajo futuro

Recoge las conclusiones, la revisión de objetivos, la aportación del proyecto, las competencias adquiridas, la valoración personal y las líneas de trabajo futuro.

El documento incluye siete anexos: especificación completa de la API REST (Anexo A), catálogo de métricas de calidad (Anexo B), backlog completo (Anexo C), plan de riesgos detallado (Anexo D), estimación de costes y viabilidad (Anexo E), manual de usuario (Anexo F) y colección de diagramas **UML! (UML!)** (Anexo G).

Capítulo 2

Antecedentes y estado del arte

LA CALIDAD de datos se ha consolidado como un campo de estudio fundamental dentro de la informática y la gestión de la información. A lo largo de las últimas tres décadas, la comunidad científica ha producido marcos conceptuales, estándares internacionales y herramientas especializadas que buscan sistematizar la evaluación y la mejora de los datos utilizados en sistemas de información. En el contexto de la IA y el ML, la calidad de los datos de entrenamiento condiciona directamente la fiabilidad de los modelos resultantes, lo que ha impulsado una línea de investigación conocida como *data-centric AI*. Este capítulo examina los fundamentos teóricos, el marco normativo, las herramientas existentes y la justificación académica que sustentan el desarrollo de DATAQUAL como plataforma de evaluación de calidad de datos para proyectos de IA.

2.1 La importancia de la calidad de datos en proyectos de IA

2.1.1 Marcos conceptuales fundacionales

Los trabajos seminales de Wang y Strong [WS96] establecieron un marco conceptual para la calidad de datos basado en cuatro categorías fundamentales:

- **Calidad intrínseca:** precisión, objetividad, credibilidad y reputación de los datos.
- **Calidad contextual:** relevancia, valor añadido, completitud, cantidad apropiada y oportunidad temporal de los datos respecto a la tarea en la que se utilizan.
- **Calidad representacional:** interpretabilidad, facilidad de comprensión, representación concisa y representación consistente.
- **Accesibilidad:** disponibilidad de los datos y seguridad de acceso a los mismos.

Este marco ha sido ampliamente adoptado tanto en la investigación como en la industria, y constituye la base sobre la que se han construido estándares internacionales como ISO/IEC 25012.

2.1.2 Impacto económico de la mala calidad de datos

Redman [Red98, Red01] cuantificó por primera vez el impacto económico de la mala calidad de datos en las organizaciones, estableciendo que las empresas gastan entre el 10 %

2. ANTECEDENTES Y ESTADO DEL ARTE

y el 25 % de sus ingresos en gestionar problemas derivados de datos de baja calidad. Estos hallazgos han sido corroborados y ampliados por estudios posteriores del MIT Information Quality Program y del Data Management Body of Knowledge (DMBOK). Según estimaciones de Gartner [Gar17], la mala calidad de datos cuesta a las organizaciones una media de 12,9 millones de dólares anuales.

En el contexto específico de proyectos de IA, este coste se amplifica de forma considerable: un modelo entrenado con datos defectuosos no solo produce resultados incorrectos, sino que las decisiones basadas en esas predicciones pueden tener consecuencias financieras, legales y reputacionales significativas. Sambasivan et al. [SKH⁺21] demostraron, en un estudio realizado en Google Research, que los problemas de calidad de datos constituyen la causa principal de fallos en sistemas de IA en producción, superando incluso a los errores algorítmicos. Su trabajo, titulado «Everyone wants to do the model work, not the data work», evidenció la brecha entre la atención que reciben los modelos y la que reciben los datos que los alimentan.

Múltiples encuestas en la industria [Kag22] coinciden en que los profesionales de datos dedican entre el 60 % y el 80 % de su tiempo a tareas de limpieza y preparación de datos, lo que refleja tanto la prevalencia de los problemas de calidad como la ausencia de herramientas automatizadas que agilicen el diagnóstico.

2.1.3 El paradigma data-centric AI

Andrew Ng [Ng21], uno de los investigadores más influyentes en IA, ha popularizado el concepto de *data-centric AI* como contraposición al enfoque tradicional *model-centric*. Mientras que el enfoque *model-centric* se centra en mejorar los algoritmos manteniendo los datos fijos, el enfoque *data-centric* propone mantener los modelos fijos y mejorar sistemáticamente los datos. Este cambio de paradigma reconoce que, para la mayoría de aplicaciones prácticas de IA, la mejora de los datos produce un impacto mayor que la mejora de los algoritmos.

Diversas iniciativas han consolidado este paradigma en la comunidad:

- La *Data-Centric AI Competition*, organizada por Andrew Ng y Landing AI, en la que los participantes compiten mejorando un dataset fijo en lugar de un modelo fijo.
- *DataPerf*, una iniciativa de MLCommons que proporciona *benchmarks* centrados en la calidad de datos para sistemas de ML.
- El área de investigación *Data Quality for AI* (DQAI), que explora métricas de calidad específicas para conjuntos de datos destinados al entrenamiento de modelos.

Además, la problemática del sesgo algorítmico refuerza la necesidad de herramientas de evaluación de calidad. Los datos desequilibrados o no representativos son la raíz de muchos casos documentados de sesgo en sistemas de IA: desde algoritmos de contratación que

discriminan por género [Das18] hasta sistemas de reconocimiento facial con tasas de error desproporcionadas para minorías étnicas [BG18]. La calidad de los datos —en particular, el equilibrio de clases y la representatividad— es un factor determinante en la equidad de los sistemas inteligentes.

2.2 Marco normativo: estándares ISO

La evaluación de la calidad de datos se enmarca dentro de un conjunto de estándares internacionales publicados por la ISO. DATAQUAL ha sido diseñado para alinearse con tres estándares clave: ISO/IEC 25012, ISO/IEC 25024 e ISO/IEC 5259.

2.2.1 ISO/IEC 25012: Modelo de calidad de datos

El estándar ISO/IEC 25012 [ISO08] forma parte de la familia SQUARE y define un modelo de calidad de datos que comprende 15 características organizadas en dos categorías principales:

- **Calidad inherente:** propiedades que pertenecen a los datos en sí mismos, independientemente del sistema que los almacena. Incluye exactitud, completitud, consistencia, credibilidad y actualidad (*currentness*).
- **Calidad dependiente del sistema:** propiedades que dependen del contexto tecnológico en el que se utilizan los datos. Incluye disponibilidad, portabilidad y recuperabilidad.

DATAQUAL implementa métricas que cubren las principales características inherentes del estándar, añadiendo además métricas específicas para proyectos de IA y ML que no están contempladas en la norma. El Cuadro 2.1 muestra la correspondencia entre las características de ISO/IEC 25012 y las métricas implementadas en DATAQUAL.

2.2.2 ISO/IEC 5259: Calidad de datos para analítica e IA

El estándar ISO/IEC 5259 [ISO24] es de reciente publicación y aborda específicamente la calidad de datos en contextos de analítica e IA. Se estructura en cuatro partes:

1. **Parte 1 — Visión general:** establece los conceptos fundamentales y el vocabulario común.
2. **Parte 2 — Métricas de calidad de datos:** define métricas cuantitativas estandarizadas para evaluar la calidad de datos destinados a sistemas de IA.
3. **Parte 3 — Gestión de datos:** cubre la gestión de conjuntos de datos, incluyendo versionado y trazabilidad.
4. **Parte 4 — Marco de procesos:** describe el flujo de evaluación integrado en el ciclo de vida de un proyecto de IA.

Cuadro 2.1: Correspondencia entre características de ISO/IEC 25012 y métricas de DATAQUAL

Característica ISO 25012	Métrica DATAQUAL	Implementación
Compleitud	completeness	Ratio de valores no nulos por columna
Exactitud	syntactic_accuracy	Conformidad con patrones de formato
Consistencia	logical_consistency	Reglas IF-THEN entre columnas
Actualidad	currentness	Antigüedad del dato más reciente
Credibilidad	uniqueness	Detección de registros duplicados
<i>Métricas adicionales específicas para IA/ML</i>		
—	outliers	Detección de valores atípicos (IQR, Z-score)
—	class_balance	Equilibrio de clases (entropía de Shannon)

DATAQUAL se alinea con este estándar en varios aspectos: proporciona métricas cuantitativas estandarizadas (Parte 2), implementa gestión de datasets con versionado (Parte 3) y ofrece un flujo de evaluación integrado en el ciclo de vida del proyecto (Parte 4).

2.2.3 ISO/IEC 25024: Medición de la calidad de datos

El estándar ISO/IEC 25024 [ISO15] complementa al 25012 proporcionando métricas concretas y fórmulas de medición para cada característica de calidad. De este modo, se pasa de un modelo conceptual abstracto a un conjunto de indicadores cuantificables.

DATAQUAL implementa directamente varias de las métricas definidas en este estándar:

- **Ratio de completitud:** proporción de valores no nulos sobre el total de valores esperados en una columna o dataset.
- **Ratio de unicidad:** proporción de valores únicos respecto al total de registros, utilizada para detectar duplicados.
- **Ratio de conformidad sintáctica:** proporción de valores que se ajustan a un patrón o formato predefinido (expresiones regulares, rangos numéricos, conjuntos de valores permitidos).

La adopción de estas fórmulas estandarizadas garantiza que los resultados de DATAQUAL sean comparables con los obtenidos por otras herramientas que implementen el mismo estándar, facilitando la interoperabilidad y la auditoría.

Además, el Reglamento Europeo de Inteligencia Artificial [Par24] exige que las organizaciones puedan demostrar la calidad y trazabilidad de los datos utilizados en sus modelos, lo que convierte la evaluación estandarizada de calidad de datos en una necesidad no solo

técnica, sino también de cumplimiento normativo.

2.3 Estado del arte: herramientas existentes

En el ecosistema actual existen diversas herramientas orientadas a la evaluación y validación de la calidad de datos. Esta sección analiza las más relevantes, examinando sus fortalezas y limitaciones para contextualizar la contribución de DATAQUAL.

2.3.1 Great Expectations

Great Expectations [Gre24] es una biblioteca de código abierto escrita en Python para la definición y validación de «expectativas» (*expectations*) sobre conjuntos de datos. Las expectativas son aserciones declarativas que describen las propiedades que se esperan de los datos, como «esta columna no debe contener valores nulos» o «los valores de esta columna deben estar comprendidos entre 0 y 100».

Fortalezas. Great Expectations cuenta con un ecosistema maduro y una comunidad activa. Soporta múltiples *backends* de ejecución (Pandas, Spark, SQL), lo que permite evaluar datasets de distintos tamaños y procedencias. Además, genera automáticamente documentación visual de los resultados de validación (*Data Docs*).

Limitaciones. Se trata de una herramienta *code-first* que requiere conocimientos de Python para su configuración y uso. No dispone de una interfaz gráfica integrada, lo que excluye a usuarios no técnicos. No ofrece Quality Gates nativos, no implementa un sistema de rastreo de issues entre ejecuciones sucesivas y presenta una curva de aprendizaje pronunciada para equipos sin experiencia en programación.

2.3.2 Apache Deequ

Apache Deequ [SLS⁺18] es una biblioteca de verificación de calidad de datos desarrollada por Amazon y construida sobre Apache Spark. Está diseñada para operar a gran escala sobre datasets masivos en entornos distribuidos.

Fortalezas. Su principal ventaja es la escalabilidad masiva que hereda de Spark, lo que permite procesar datasets de millones de registros de forma distribuida. Ofrece integración nativa con los servicios de Amazon Web Services, verificación declarativa de restricciones y detección de anomalías basada en series temporales.

Limitaciones. Requiere un entorno JVM y un clúster de Spark para su ejecución, lo que supone una barrera de entrada considerable. No dispone de interfaz web, resulta desproporcionado para datasets de tamaño medio y no implementa métricas específicas para ML como

2. ANTECEDENTES Y ESTADO DEL ARTE

el equilibrio de clases.

2.3.3 DQOps

DQOps [DQO24] es una herramienta de calidad de datos de código abierto inspirada en la filosofía de SonarQube [Son24]. Proporciona un enfoque sistemático a la calidad de datos con monitorización continua y alertas automatizadas.

Fortalezas. DQOps comparte una filosofía similar a la de DATAQUAL en cuanto al tratamiento de la calidad de datos como un proceso continuo. Soporta múltiples fuentes de datos (BigQuery, Snowflake, PostgreSQL), ofrece una interfaz web para la configuración y visualización de resultados, y permite definir alertas automatizadas ante degradaciones de calidad.

Limitaciones. Está orientado fundamentalmente a bases de datos y no a ficheros de datos (datasets en formatos CSV, XLSX o Parquet). Su configuración es compleja, no ha sido diseñado específicamente para proyectos de IA/ML y carece de métricas como el equilibrio de clases o la detección de outliers.

2.3.4 Herramientas comerciales

Existen también soluciones comerciales orientadas a la preparación de datos, como Trifacta, Alteryx y Talend. Estas herramientas ofrecen interfaces gráficas pulidas, capacidades de transformación de datos y soporte empresarial.

No obstante, presentan limitaciones significativas para el caso de uso de DATAQUAL: su coste de licenciamiento es elevado, generan dependencia del proveedor (*vendor lock-in*), están orientadas a la preparación y transformación de datos (*data wrangling*) más que a la evaluación de calidad para IA, y muchas de ellas operan como servicios en la nube (Software as a Service (SAAS)), lo que compromete la soberanía de los datos.

2.3.5 Análisis comparativo

El Cuadro 2.2 presenta una comparación sistemática de DATAQUAL con las tres herramientas de código abierto analizadas. Se han seleccionado quince características que cubren la funcionalidad, la accesibilidad, las capacidades específicas para IA/ML y los aspectos operativos.

Como se puede observar, DATAQUAL ocupa un nicho diferenciado al combinar una interfaz web completa con métricas específicas para IA/ML, fingerprinting de issues y auto-alojamiento. Si bien Great Expectations supera ampliamente a DATAQUAL en el número de métricas predefinidas, su naturaleza *code-first* limita su accesibilidad. Apache Deequ ofrece una escalabilidad superior, pero a costa de una complejidad operativa que lo hace inviable

Cuadro 2.2: Comparativa de DATAQUAL con herramientas de calidad de datos existentes

Característica	DATAQUAL	Great Exp.	Deequ	DQOps
Interfaz web	Sí	No	No	Sí
Requiere programación	No	Sí	Sí	Parcial
Métricas predefinidas	7	300+	20+	150+
Detección automática de tipos	Sí	No	No	Parcial
Quality Gates	Sí	No	No	Sí
Fingerprinting de issues	Sí	No	No	No
Comparación entre ejecuciones	Sí	No	Sí	Sí
Equilibrio de clases	Sí	No	No	No
Detección de outliers	Sí	Parcial	Sí	Parcial
Enmascaramiento de PII	Sí	No	No	No
Versionado de datasets	Sí	No	No	No
Procesamiento asíncrono	Sí	No	Sí	Sí
Autoalojable	Sí	Sí	Sí	Sí
Específico para IA/ML	Sí	No	No	No
Coste	Gratuito	Gratuito	Gratuito	Freemium

para equipos pequeños o datasets de tamaño moderado. DQOps comparte la filosofía de calidad continua, pero su orientación a bases de datos y la ausencia de métricas específicas para ML lo sitúan en un segmento diferente.

2.4 Diferenciación de DATAQUAL

A partir del análisis comparativo anterior, se identifican siete factores clave que diferencian a DATAQUAL de las herramientas existentes:

1. **Solución *full-stack* con interfaz integrada.** A diferencia de herramientas *code-only* como Great Expectations o Apache Deequ, DATAQUAL proporciona una interfaz web completa construida con React [Met24] y Next.js [Ver24] que permite a usuarios no técnicos configurar métricas, lanzar evaluaciones y consultar resultados sin escribir código.
2. **Diseño centrado en IA/ML.** DATAQUAL incorpora métricas específicas para proyectos de aprendizaje automático que no se encuentran en las herramientas generalistas: equilibrio de clases medido mediante entropía de Shannon y evaluación de la actualidad temporal de los datos (*currentness*).
3. **Sistema de fingerprinting SHA-256.** Inspirado en SonarQube [Son24], DATAQUAL genera un hash SHA-256 para cada issue detectado, lo que permite rastrear su ciclo de vida a lo largo de múltiples ejecuciones: issues nuevos, recurrentes y corregidos.
4. **Quality Gates como puntos de decisión.** Cada evaluación produce un veredicto global (PASSED, WARNING o FAILED) basado en umbrales configurables, proporcionando una señal inequívoca sobre la aptitud del dataset para su uso.

2. ANTECEDENTES Y ESTADO DEL ARTE

5. **Protección de PII integrada.** El enmascaramiento de columnas sensibles es una funcionalidad nativa de la plataforma, no un complemento externo. Esto facilita el cumplimiento del RGPD y otras normativas de protección de datos.
6. **Arquitectura de plugins extensible.** El motor de métricas se basa en una clase abstracta `BaseMetric` y un registro global (`METRIC_REGISTRY`) que permiten añadir nuevas métricas sin modificar el código existente, siguiendo el principio abierto-cerrado de los patrones de diseño [GHJV94].
7. **Autoalojamiento con soberanía de datos.** DATAQUAL se despliega mediante Docker Compose [Doc24] sin dependencias de servicios en la nube. Todos los datos permanecen en la infraestructura del usuario, lo que garantiza la soberanía de los datos y facilita el cumplimiento normativo.

2.5 Justificación académica

El desarrollo de DATAQUAL integra de manera transversal múltiples áreas del plan de estudios del Grado en Ingeniería Informática, lo que le confiere un carácter integrador propio de un TFG. A continuación se detalla la correspondencia entre las áreas curriculares y los componentes de la plataforma.

2.5.1 Ingeniería del Software

El proyecto aplica principios de arquitectura de software a gran escala: arquitectura de microservicios orquestados con Docker Compose [Doc24], patrón Model-View-Controller (MVC) en el backend Flask [Ron24], y patrones de diseño clásicos [GHJV94] como Abstract Factory (registro de métricas) y Observer (sistema de eventos de evaluación). Se ha seguido control de versiones con Git y metodologías ágiles de desarrollo.

2.5.2 Sistemas distribuidos

La plataforma implementa procesamiento asíncrono mediante colas de mensajes con Celery [Cel24] y Redis [Red24] como *broker* de mensajes. La orquestación de los ocho servicios Docker incluye *healthchecks*, reinicio automático y comunicación inter-servicio mediante redes internas. Flower proporciona monitorización en tiempo real de los *workers* asíncronos.

2.5.3 Bases de datos

El esquema relacional en PostgreSQL [The24] incluye el uso de campos `JSONB` para configuraciones flexibles de métricas, migraciones gestionadas con Flask-Migrate/Alembic y almacenamiento de objetos compatible con S3 mediante MinIO [Min24]. El Object-Relational Mapping (ORM) SQLAlchemy [Bay24] proporciona la capa de abstracción sobre la base de datos relacional.

2.5.4 Desarrollo web

DATAQUAL es una aplicación *full-stack* que combina un *frontend* construido con React [Met24], Next.js [Ver24], TypeScript [Mic24] y Material User Interface (UI) [MUI24], con un backend RESTful implementado en Flask [Ron24]. La autenticación se gestiona mediante tokens JWT, y la comunicación entre frontend y backend se realiza a través de una API REST.

2.5.5 Análisis de datos y estadística

El motor de métricas implementa técnicas estadísticas como el rango intercuartílico (IQR) y el Z-score para la detección de outliers, la entropía de Shannon para la evaluación del equilibrio de clases, y el Exploratory Data Analysis (EDA) interactivo con histogramas, diagramas de caja, matrices de correlación y detección de anomalías.

2.5.6 Seguridad informática

La plataforma incorpora múltiples capas de seguridad: autenticación y autorización mediante JWT, derivación de contraseñas con Password-Based Key Derivation Function 2 (PBKDF2), limitación de tasa de peticiones (*rate limiting*), validación de entradas en el servidor, y enmascaramiento de PII para proteger datos sensibles conforme al RGPD.

2.5.7 DevOps y despliegue

La containerización con Docker [Doc24] y la orquestación con Docker Compose permiten desplegar la plataforma completa con un único comando. La configuración basada en variables de entorno facilita la adaptación a distintos entornos (desarrollo, pruebas, producción), y Flower proporciona un panel de monitorización para los *workers* de Celery.

En conjunto, el desarrollo de DATAQUAL demuestra la capacidad de integrar conocimientos de múltiples disciplinas informáticas en un producto de software funcional, cohesionado y alineado con estándares internacionales, lo que constituye el objetivo fundamental de un TFG en Ingeniería Informática.

Capítulo 3

Objetivos

UNA vez analizado el contexto teórico, el marco normativo y las herramientas existentes en el capítulo 2, es posible delimitar con precisión los objetivos que guían el desarrollo de DATAQUAL. Este capítulo define, en primer lugar, el objetivo general del TFG y, a continuación, lo descompone en cinco objetivos específicos que abordan las principales dimensiones funcionales de la plataforma. Se describen, además, los objetivos transversales de calidad del software que condicionan las decisiones arquitectónicas y, por último, se establece el alcance del proyecto junto con sus limitaciones.

3.1 Objetivo general

El objetivo general de este TFG consiste en **diseñar, implementar y validar una plataforma web interactiva y una API RESTful que permitan evaluar de forma automatizada la calidad de los datos empleados en proyectos de IA, ML y minería de datos**, proporcionando métricas cuantitativas, visualizaciones interactivas y mecanismos de diagnóstico que faciliten la identificación y corrección de problemas de calidad en los conjuntos de datos.

Este objetivo responde a la necesidad, identificada en la sección 1.1, de disponer de herramientas que superen las limitaciones de los enfoques ad hoc —*scripts* puntuales, consultas SQL manuales o inspecciones en cuadernos Jupyter —y que, al mismo tiempo, no impongan la barrera técnica que presentan las bibliotecas basadas exclusivamente en código (sección 2.3). DATAQUAL aspira a ofrecer una solución *full-stack*, alineada con los estándares ISO/IEC 25012 [ISO08] e ISO/IEC 5259 [ISO24], que democratice la evaluación de calidad de datos haciéndola accesible tanto a perfiles técnicos como a usuarios de negocio.

3.2 Objetivos específicos

El objetivo general se descompone en los siguientes cinco objetivos específicos (**OE1–OE5**), cada uno de los cuales aborda una dimensión funcional diferenciada de la plataforma.

3.2.1 OE1. Módulo de ingesta y almacenamiento

Enunciado. Desarrollar un módulo de ingesta y almacenamiento que permita la carga, validación de formato y persistencia segura de conjuntos de datos.

Este objetivo abarca los siguientes requisitos funcionales:

- **Soporte multiformato.** La plataforma debe aceptar conjuntos de datos en los formatos más utilizados en proyectos de IA: CSV, XLSX, JSON y Parquet.
- **Parseo robusto.** Para archivos CSV, el módulo debe implementar una estrategia de *fallback* que combine múltiples codificaciones (UTF-8, UTF-8-BOM, CP1252, Latin-1), separadores (coma, punto y coma, tabulador, *pipe*) y motores de lectura, garantizando la ingesta correcta del mayor número posible de archivos reales.
- **Almacenamiento seguro.** Los archivos originales deben almacenarse en un sistema de almacenamiento de objetos compatible con S3 (MinIO), mientras que los metadatos —esquema, estadísticas descriptivas, conteos— se persisten en PostgreSQL.
- **Extracción automática de esquema.** El sistema debe inferir automáticamente los nombres de columna, los tipos de dato y las estadísticas descriptivas básicas (media, mediana, desviación estándar, mínimo, máximo) de cada conjunto de datos cargado.
- **Versionado de conjuntos de datos.** El módulo debe soportar la carga de nuevas versiones de un mismo conjunto de datos, manteniendo la trazabilidad entre versiones mediante una relación padre-hijo.

3.2.2 OE2. Motor de evaluación con métricas parametrizables

Enunciado. Implementar un motor de evaluación que calcule métricas de calidad estándar sobre los conjuntos de datos cargados, permitiendo su parametrización según la tipología del proyecto.

Las métricas implementadas deben cubrir las principales dimensiones de calidad definidas en ISO/IEC 25012 [ISO08] y en ISO/IEC 5259 [ISO24], complementadas con métricas específicas para el contexto de IA y ML. En concreto, el motor debe incluir las siguientes **seis métricas evaluables** (que contribuyen al *Quality Score*):

1. **Compleitud** (*completeness*): ratio de valores no nulos por columna, alineada con la característica de completitud de ISO/IEC 25012.
2. **Exactitud sintáctica** (*syntactic_accuracy*): conformidad de los valores con patrones de formato definidos mediante expresiones regulares, correspondiente a la dimensión de exactitud.
3. **Consistencia lógica** (*logical_consistency*): verificación de reglas condicionales entre columnas (reglas IF - THEN), alineada con la dimensión de consistencia.

3. OBJETIVOS

4. **Actualidad** (*currentness*): antigüedad del dato más reciente respecto a un instante de referencia, correspondiente a la dimensión de actualidad (*currentness*).
5. **Unicidad** (*uniqueness*): detección de registros duplicados, relacionada con la credibilidad de los datos.
6. **Equilibrio de clases** (*class_balance*): evaluación del balance de la variable objetivo mediante la entropía de Shannon, métrica específica para proyectos de ML que permite detectar desequilibrios que sesgan los modelos [Ng21].

Adicionalmente, el motor debe incluir una clase de **detección de valores atípicos** (*outliers*) basada en IQR [Tuk77] y Z-score. Siguiendo la recomendación de ISO/IEC 5259 [ISO24] de no tratar los *outliers* como una dimensión de calidad per se, esta clase **no participará en el *Quality Score*** sino que se utilizará exclusivamente como herramienta de diagnóstico en el módulo de perfilado de datos (*EDA*).

El motor debe adoptar una arquitectura de *plugins*, de modo que cada métrica se implemente como una clase independiente que herede de una clase base abstracta y se registre dinámicamente en un registro centralizado. Esta arquitectura garantiza la extensibilidad del sistema, permitiendo incorporar nuevas métricas sin modificar el núcleo del motor.

Adicionalmente, cada métrica debe ser parametrizable: umbrales de aceptación, columnas objetivo, métodos de detección y pesos relativos (en una escala de 0,0 a 1,0) que determinen su contribución a la puntuación global de calidad. El sistema debe soportar también plantillas de métricas que permitan reutilizar configuraciones entre proyectos.

3.2.3 OE3. API RESTful documentada

Enunciado. Diseñar e implementar una API RESTful que exponga *endpoints* para la carga de datos, configuración de evaluaciones, ejecución de análisis y consulta de resultados, permitiendo la integración con flujos de trabajo externos.

La API debe organizarse en módulos funcionales cohesivos, siguiendo el patrón *Blueprint* de Flask:

auth Registro de usuarios, inicio de sesión, refresco de *tokens* JWT y cierre de sesión.

projects

Operaciones Create, Read, Update, Delete (CRUD) sobre proyectos, incluyendo la asignación de conjuntos de datos y la configuración de métricas.

datasets

Carga de conjuntos de datos, consulta de metadatos, gestión de versiones y descarga de archivos.

metrics

Definición y configuración de métricas de calidad, gestión de plantillas y consulta del

catálogo de métricas disponibles.

evaluations

Lanzamiento de evaluaciones, consulta de progreso, recuperación de resultados e *issues* detectados.

admin

Utilidades de administración del sistema, incluyendo un *endpoint* de comprobación de salud (*health check*).

Todos los *endpoints* protegidos deben requerir autenticación mediante JWT. El formato de respuesta debe ser uniforme y predecible, siguiendo la estructura {success, data, message}, con soporte de paginación en los listados. Estos seis módulos funcionales se implementan como *blueprints* Flask principales; la convención REST de rutas anidadas puede requerir *blueprints* adicionales de organización técnica (sub-rutas de proyecto y vistas de agregación) sin que ello amplíe el alcance funcional del módulo.

3.2.4 OE4. Aplicación web con *dashboards* interactivos

Enunciado. Desarrollar una aplicación web para la gestión de proyectos y conjuntos de datos, que permita configurar evaluaciones y umbrales de calidad personalizados, consultar el historial de evaluaciones realizadas y visualizar las métricas obtenidas mediante cuadros de mando interactivos.

La interfaz de usuario debe proporcionar las siguientes capacidades:

- **Cuadro de mando principal** con resumen de proyectos, conjuntos de datos y distribución de *Quality Gates*.
- **Página de detalle de proyecto** con pestañas diferenciadas para conjuntos de datos, configuración de métricas, historial de evaluaciones y estado del *Quality Gate*.
- **Visualización de resultados de evaluación** con indicadores de puntuación de calidad (*gauge*), pestañas por métrica y listado de *issues* detectados.
- **Exploración interactiva de datos** (EDA): histogramas, diagramas de caja, gráficos de barras, matrices de correlación (Pearson y Spearman) y detección visual de valores atípicos con factor IQR configurable.
- **Gráficos de tendencia** que muestren la evolución de la calidad a lo largo de las sucesivas evaluaciones.
- **Indicadores visuales de *Quality Gate*** con códigos de color: verde (PASSED), amarillo (WARNING) y rojo (FAILED).
- **Diseño adaptable** (*responsive*) con soporte para dispositivos de escritorio, tableta y teléfono móvil.

3. OBJETIVOS

3.2.5 OE5. Mecanismos de identificación de problemas

Enunciado. Desarrollar mecanismos de identificación y diagnóstico de problemas de calidad que permitan localizar registros defectuosos en los conjuntos de datos, clasificar las incidencias por severidad y rastrear su evolución entre evaluaciones sucesivas.

Este objetivo se materializa en tres componentes principales:

1. **Sistema de *issues*.** Cada problema de calidad detectado debe registrarse como un *issue* con cuatro niveles de severidad (*critical*, *high*, *medium*, *low*), calculada dinámicamente en función de la distancia al umbral configurado. Cada *issue* debe incluir muestras de los datos afectados (hasta cinco filas de ejemplo), con enmascaramiento automático de columnas marcadas como PII.
2. ***Fingerprinting* determinista.** Cada *issue* debe recibir un identificador único generado mediante SHA-256 a partir de sus atributos característicos (métrica, columna, tipo de problema), permitiendo su identificación unívoca entre ejecuciones independientes. Este mecanismo, inspirado en el sistema de rastreo de *issues* de SonarQube [Son24], permite clasificar automáticamente cada incidencia como *nueva*, *recurrente* o *corregida* respecto a la evaluación anterior.
3. ***Quality Gates*.** Umbrales mínimos configurables que determinan si un conjunto de datos es apto para su uso, con tres estados posibles: PASSED (todos los umbrales superados), WARNING (margen estrecho respecto a algún umbral) y FAILED (al menos un umbral incumplido). Las *Quality Gates* proporcionan una señal clara y automatizable que puede integrarse en *pipelines* de MLOPS [KKH23] o de integración continua para bloquear el uso de conjuntos de datos de calidad insuficiente.

3.3 Objetivos transversales

Además de los objetivos funcionales descritos, el desarrollo de DATAQUAL persigue un conjunto de objetivos transversales de calidad del software que condicionan las decisiones arquitectónicas y tecnológicas del proyecto:

Seguridad

Autenticación basada en JWT con *tokens* de acceso y refresco, *hashing* de contraseñas mediante PBKDF2, limitación de tasa de peticiones (*rate limiting*), validación exhaustiva de las entradas del usuario, protección contra inyección de código en las reglas de consistencia lógica y enmascaramiento automático de columnas marcadas como PII.

Escalabilidad

Procesamiento asíncrono de evaluaciones mediante Celery y Redis como *broker* de mensajes, lo que permite escalar horizontalmente el número de *workers* en función de la carga de trabajo, así como separación de responsabilidades entre servicios.

Mantenibilidad

Arquitectura modular basada en *blueprints*, servicios y modelos, con un sistema de métricas extensible por *plugins* que permite incorporar nuevas métricas con un esfuerzo mínimo. Migraciones de base de datos versionadas y separación estricta entre *frontend* y *backend*.

Usabilidad

Interfaz de usuario basada en Material Design con formularios dotados de validación en tiempo real, notificaciones *toast*, migas de pan (*breadcrumbs*) de navegación y estados de carga y error claramente diferenciados.

Portabilidad

Despliegue del *stack* completo mediante Docker Compose, con configuración por variables de entorno y sin dependencias de servicios en la nube propietarios. Los ocho servicios que componen la plataforma —*frontend*, *backend*, PostgreSQL, MinIO, Redis, Celery *worker*, Celery Beat y Flower— se orquestan de forma conjunta, garantizando la reproducibilidad del entorno y la soberanía sobre los datos.

3.4 Alcance y limitaciones

Para delimitar con claridad el ámbito del presente TFG, resulta necesario establecer tanto lo que forma parte del alcance del proyecto como aquello que queda excluido.

3.4.1 Dentro del alcance

- Diseño e implementación de la plataforma DATAQUAL como aplicación web *full-stack* con *frontend* en Next.js y *backend* en Flask.
- Implementación de las siete métricas de calidad descritas en el objetivo OE2 (sección 3.2.2), con parametrización completa y sistema de pesos.
- Desarrollo de la API RESTful con los seis módulos definidos en el objetivo OE3 (sección 3.2.3).
- Despliegue contenerizado del *stack* completo mediante Docker Compose.
- Validación de la plataforma con conjuntos de datos reales, verificando el correcto funcionamiento de las métricas, las *Quality Gates* y el sistema de *fingerprinting*.
- Documentación técnica: especificación de la API y catálogo de métricas (anexos del presente documento).

3.4.2 Fuera del alcance

- **Procesamiento de *big data*.** La plataforma está diseñada para conjuntos de datos de tamaño moderado (hasta 100 MB por archivo). La evaluación de conjuntos de datos masivos que requieran procesamiento distribuido (Apache Spark, Dask) queda fuera del alcance.

3. OBJETIVOS

- **Evaluación en tiempo real** (*streaming*). DATAQUAL opera sobre conjuntos de datos estáticos cargados por el usuario. La evaluación continua de flujos de datos en tiempo real no forma parte del alcance actual.
- **Corrección automática de datos**. La plataforma identifica y diagnostica problemas de calidad, pero no implementa mecanismos de corrección automática de los datos (limpieza, imputación, deduplicación).
- **Integración nativa con plataformas en la nube**. Aunque la arquitectura basada en MinIO es compatible con S3, no se desarrollan conectores específicos para servicios como AWS, Google Cloud o Azure.
- **Internacionalización**. La interfaz de usuario se desarrolla únicamente en español, sin soporte multiidioma.

3.4.3 Restricciones tecnológicas

El desarrollo del proyecto está sujeto a las siguientes restricciones tecnológicas, derivadas tanto de requisitos académicos como de decisiones arquitectónicas:

- El *backend* se implementa en Python con el *framework* Flask, utilizando PostgreSQL como sistema de gestión de bases de datos relacional.
- El *frontend* se desarrolla con React y Next.js, utilizando TypeScript como lenguaje principal.
- El procesamiento asíncrono se delega en Celery con Redis como *broker* de mensajes.
- Todo el sistema debe poder desplegarse mediante Docker Compose en una única máquina, sin requerir infraestructura distribuida.
- El proyecto debe alinearse con los estándares ISO/IEC 25012 [ISO08], ISO/IEC 25024 [ISO15] e ISO/IEC 5259 [ISO24] en lo relativo a las dimensiones y métricas de calidad de datos implementadas.

Capítulo 4

Metodología y herramientas

ESTE capítulo describe el enfoque metodológico completo adoptado para el desarrollo de DATAQUAL: la metodología de gestión del proyecto (Scrum adaptado), la metodología de desarrollo software, la pila tecnológica, la planificación temporal con diagrama de Gantt, la estimación de esfuerzo, el análisis de costes y la gestión de riesgos del proyecto.

4.1 Gestión del proyecto

4.1.1 Metodología ágil adoptada: Scrum adaptado

Para la gestión del proyecto se ha adoptado **Scrum adaptado a un TFG individual** [SS20], metodología ágil iterativa e incremental que estructura el trabajo en sprints de duración fija y favorece la entrega continua de valor. La justificación de esta elección se apoya en tres argumentos principales:

- **Adecuación al tipo de proyecto.** DATAQUAL es un proyecto de desarrollo de software con requisitos que evolucionan conforme se profundiza en el dominio (calidad de datos, estándares ISO). Scrum permite incorporar aprendizajes del dominio en el backlog de forma natural.
- **Retroalimentación periódica de tutores.** Las reuniones quincenales con los tutores actúan como sprint reviews, proporcionando retroalimentación continua sobre dirección técnica y calidad del trabajo.
- **Trazabilidad del avance.** El uso de GitHub Issues como herramienta de backlog permite mantener un historial auditado del progreso, esencial para la redacción de la memoria.

4.1.2 Adaptación de Scrum al TFG individual

La metodología Scrum estándar ha sido adaptada a la naturaleza unipersonal y académica del proyecto de la siguiente forma:

Sprints

El proyecto se divide en **6 sprints** de entre dos y cuatro semanas de duración, cada uno orientado a un conjunto coherente de funcionalidades (véase la planificación en la Sección 4.6).

4. METODOLOGÍA Y HERRAMIENTAS

Roles

Al ser un proyecto individual, el estudiante asume simultáneamente los tres roles de Scrum:

- *Product Owner*: define y prioriza el backlog en función de los objetivos del TFG y de la retroalimentación de los tutores.
- *Scrum Master*: vela por la aplicación de la metodología y elimina impedimentos técnicos.
- *Developer*: implementa las historias de usuario seleccionadas para cada sprint.

Sprint planning

Al inicio de cada sprint se define el objetivo, se seleccionan las historias de usuario del backlog y se descomponen en tareas concretas registradas en GitHub Issues.

Daily standup

Sustituido por una **revisión personal diaria** informal: el desarrollador evalúa el progreso del día anterior y planifica las tareas del día.

Sprint review

Reunión quincenal con los tutores en la que se presenta el incremento funcional, se obtiene retroalimentación y se ajusta el backlog si procede.

Sprint retrospective

Reflexión escrita al final de cada sprint sobre qué ha ido bien, qué ha ido mal y qué mejorar, documentada en la memoria (véase Capítulo 5).

4.1.3 Herramientas de gestión

GitHub Issues

Herramienta principal de gestión del backlog. Cada historia de usuario y tarea técnica se registra como una *issue* con etiquetas de prioridad, estimación en horas y sprint asignado. Los *milestones* de GitHub corresponden a los sprints del proyecto.

GitHub Projects (Kanban board)

Tablero Kanban con columnas *Backlog*, *Sprint activo*, *En progreso*, *En revisión* y *Cerrado*, utilizado para visualizar el flujo de trabajo dentro de cada sprint.

Git + GitHub

Control de versiones con ramas de funcionalidad (*feature branches*) por historia de usuario y *merges* a main tras revisión.

4.2 Metodología de desarrollo

El desarrollo de DATAQUAL ha seguido una **metodología iterativa e incremental**, donde cada sprint produce un incremento funcional desplegable que amplía las capacidades de la plataforma. Los principios subyacentes —entrega frecuente, retroalimentación continua y adaptación del plan ante nuevos requisitos —han guiado todo el proceso de desarrollo.

4.2.1 Desarrollo iterativo

El proyecto se ha organizado en **iteraciones** de duración variable (entre una y tres semanas), cada una de las cuales ha producido un incremento funcional desplegable. Cada iteración ha comprendido las siguientes actividades:

1. **Planificación:** selección de las funcionalidades a implementar en la iteración, priorizando aquellas que aportan mayor valor o que desbloquean funcionalidades posteriores.
2. **Diseño:** definición de la arquitectura y el modelo de datos necesarios para las funcionalidades seleccionadas, tomando como referencia los estándares ISO/IEC 25012 [ISO08] e ISO/IEC 5259 [ISO24] para el diseño de las métricas de calidad.
3. **Implementación:** desarrollo del código del *backend* y del *frontend* de forma conjunta, garantizando que cada incremento sea funcional de extremo a extremo.
4. **Verificación:** ejecución de pruebas manuales y automatizadas para validar el correcto funcionamiento del incremento.
5. **Revisión con los tutores:** presentación del avance a los tutores del TFG para obtener retroalimentación y ajustar la dirección del desarrollo en caso necesario.

4.2.2 Prototipado

La interfaz de usuario se ha desarrollado siguiendo un enfoque de **prototipado evolutivo**: cada pantalla se ha construido inicialmente con la funcionalidad mínima necesaria y se ha refinado en iteraciones sucesivas, incorporando mejoras de usabilidad, visualizaciones adicionales y tratamiento de casos extremos. Este enfoque ha permitido obtener retroalimentación temprana sobre la experiencia de usuario sin invertir tiempo excesivo en el diseño visual previo a la validación funcional.

4.2.3 Control de versiones

El código fuente del proyecto se gestiona en un repositorio Git alojado en GitHub. Se ha seguido una estrategia de ramificación basada en una rama principal (*main*) y ramas de funcionalidad (*feature branches*) para el desarrollo de cada módulo o característica significativa. Los *commits* siguen una convención de mensajes descriptivos que facilita la trazabilidad de los cambios.

4.3 Pila tecnológica

La selección de las tecnologías que componen la plataforma responde a tres criterios principales: (1) adecuación al tipo de problema —evaluación de calidad de datos, procesamiento asíncrono, visualización interactiva—; (2) madurez y soporte comunitario de las herramientas; y (3) coherencia con las competencias adquiridas a lo largo del grado en Ingeniería Informática.

4.3.1 *Frontend*

Cuadro 4.1: Tecnologías del *frontend*

Tecnología	Versión	Justificación
Next.js	13.3	<i>Framework</i> React con enrutamiento basado en ficheros, reescritura de rutas para <i>proxy</i> al <i>backend</i> y modo <i>standalone</i> para Docker.
React	18	Biblioteca estándar para interfaces de usuario. Los <i>hooks</i> y la Context API eliminan la necesidad de gestores de estado externos como Redux.
TypeScript	5.0	Tipado estático que detecta errores en tiempo de compilación, esencial para mantener un <i>frontend</i> con más de cuarenta componentes.
Material UI	5.12	Biblioteca de componentes con <i>theming</i> , diseño adaptable y accesibilidad incorporada.
Chart.js	4.2	Biblioteca de gráficos ligera con soporte para histogramas, diagramas de dispersión y de líneas. Se utiliza mediante el <i>wrapper</i> <code>react-chartjs-2</code> .
Axios	1.3	Cliente HTTP con interceptores para inyección automática de JWT y gestión centralizada de errores.

4.3.2 *Backend*

4.3.3 Base de datos y almacenamiento

4.3.4 Procesamiento asíncrono y orquestación

4.4 Entorno de desarrollo

El desarrollo del proyecto se ha llevado a cabo en el siguiente entorno:

Editor de código

Visual Studio Code, con extensiones para Python, TypeScript, Docker y $\text{\LaTeX}$.

Control de versiones

Git 2.x con repositorio remoto en GitHub.

Entorno de ejecución local

Docker Desktop sobre Windows 11, con Docker Compose para levantar los ocho servicios de forma simultánea.

Gestor de paquetes *frontend*

pnpm, seleccionado por su eficiencia en el uso de disco y su velocidad de instalación frente a npm.

Gestor de paquetes *backend*

pip con fichero `requirements.txt` para la gestión de dependencias Python.

Cuadro 4.2: Tecnologías del *backend*

Tecnología	Versión	Justificación
Python	3.10	Lenguaje estándar en ciencia de datos e IA, con excelente integración con Pandas y NumPy para el cálculo de métricas.
Flask	2.2	<i>Framework</i> web minimalista y flexible, ideal para APIs REST. No impone una estructura fija, lo que permite una arquitectura a medida.
SQLAlchemy	2.0	ORM maduro con soporte completo para PostgreSQL, incluyendo campos JSONB, tipos ENUM y <i>pool</i> de conexiones integrado.
Flask-JWT-Extended	4.4	Autenticación JWT con <i>tokens</i> de acceso y refresco, <i>blacklisting</i> y gestión de errores personalizada.
Flask-Migrate	4.0	Migraciones de base de datos versionadas con Alembic, que permiten evolucionar el esquema de forma controlada.
Pandas	1.5	Manipulación de datos tabulares: lectura de conjuntos de datos, cálculo de métricas y generación de estadísticas descriptivas.
Gunicorn	20.1	Servidor WSGI de producción con soporte multiproceso para gestionar la concurrencia.

Cuadro 4.3: Tecnologías de base de datos y almacenamiento

Tecnología	Versión	Justificación
PostgreSQL	14	Sistema de gestión de bases de datos relacional con soporte nativo para JSONB, tipos ENUM e índices avanzados. Permite combinar datos estructurados (relaciones entre entidades) con datos semiestructurados (configuraciones, resultados).
MinIO	<i>latest</i>	Almacenamiento de objetos compatible con S3, autoalojado. Separa los archivos binarios de los datos relacionales y garantiza la soberanía de datos sin dependencia de servicios en la nube.
Redis	6	Almacén en memoria con rol dual: <i>broker</i> de mensajes para Celery y <i>storage</i> para el limitador de tasa de peticiones. Su baja latencia es idónea para la gestión de colas.

Cuadro 4.4: Tecnologías de procesamiento asíncrono y orquestación

Tecnología	Versión	Justificación
Celery	5.2	Cola de tareas distribuida estándar en Python, con soporte para reintentos, <i>timeouts</i> y seguimiento de progreso. Escalable horizontalmente mediante la adición de <i>workers</i> [Cel24].
Flower	1.2	Cuadro de mando web para la monitorización en tiempo real de <i>workers</i> , tareas y colas de Celery.
Docker	—	Contenedorización de cada servicio, garantizando aislamiento de dependencias, portabilidad y reproducibilidad [Doc24].
Docker Compose	—	Orquestación de los ocho servicios con <i>healthchecks</i> , volúmenes persistentes, red interna y variables de entorno. Un único comando (<code>docker-compose up</code>) despliega el <i>stack</i> completo.

Base de datos de desarrollo

Instancia local de PostgreSQL 14 ejecutada en contenedor Docker, con volumen persistente para preservar los datos entre reinicios.

Pruebas

pytest como *framework* de pruebas del *backend*, con soporte para pruebas unitarias y de integración.

4.5 Estrategia de pruebas

La verificación del correcto funcionamiento de DATAQUAL se ha apoyado en una combinación de pruebas automatizadas y manuales.

4.5.1 Pruebas automatizadas

Las pruebas automatizadas se implementan con **pytest** y se organizan en el directorio `backend/tests/`. Los principales tipos de pruebas cubren:

- **Pruebas unitarias** de las métricas de calidad, verificando que cada métrica produce los resultados esperados ante conjuntos de datos de prueba controlados.
- **Pruebas de integración** del sistema de versionado de conjuntos de datos, validando la correcta relación padre-hijo entre versiones y la actualización del indicador `is_latest`.
- **Pruebas de *Quality Gates***, verificando que los umbrales configurados producen los estados `PASSED`, `WARNING` y `FAILED` esperados.

La ejecución de las pruebas se realiza mediante el comando `pytest` desde el directorio `backend/`, pudiendo generar informes de cobertura con la opción `--cov`.

4.5.2 Pruebas manuales

Adicionalmente, se han diseñado guías de pruebas manuales que cubren los flujos completos de la plataforma: registro de usuario, creación de proyecto, carga de conjunto de datos, configuración de métricas, ejecución de evaluaciones y consulta de resultados. Estas pruebas verifican la integración de todos los componentes del sistema en un escenario realista.

4.6 Planificación temporal

El desarrollo del proyecto se ha distribuido a lo largo del curso académico 2025–2026, organizado en las fases que se describen a continuación.

4.6.1 Fase 1: Análisis y diseño inicial (octubre–noviembre 2025)

Estudio del estado del arte en evaluación de calidad de datos, análisis de los estándares ISO/IEC 25012 [ISO08] e ISO/IEC 5259 [ISO24], definición de los requisitos funcionales y diseño de la arquitectura general del sistema (servicios, modelo de datos, API).

4.6.2 Fase 2: Implementación del núcleo (diciembre 2025–enero 2026)

Desarrollo del *backend* base: modelos de datos, sistema de autenticación JWT, módulo de ingesta de conjuntos de datos con soporte multiformato y almacenamiento en MinIO, y esqueleto de la API RESTful.

4.6.3 Fase 3: Motor de métricas (febrero 2026)

Diseño e implementación del motor de métricas con arquitectura de *plugins*: clase base abstracta, registro dinámico e implementación de las seis métricas de calidad evaluables (completitud, exactitud sintáctica, consistencia lógica, actualidad, unicidad y equilibrio de clases) y la clase de detección de valores atípicos para el módulo de perfilado.

4.6.4 Fase 4: Interfaz de usuario y visualización (marzo 2026)

Desarrollo del *frontend* con Next.js y React: cuadro de mando principal, páginas de gestión de proyectos y conjuntos de datos, cuadros de mando de resultados de evaluación con gráficos interactivos, y sistema de exploración de datos (EDA).

4.6.5 Fase 5: Funcionalidades avanzadas (abril 2026)

Implementación del sistema de *fingerprinting* de *issues*, *Quality Gates*, comparación con línea base, enmascaramiento de PII, plantillas de métricas y refinamiento de la interfaz de usuario.

4.6.6 Fase 6: Documentación y defensa (mayo–junio 2026)

Redacción de la memoria del TFG, preparación de los anexos técnicos (especificación de la API y catálogo de métricas), validación final de la plataforma con conjuntos de datos

4. METODOLOGÍA Y HERRAMIENTAS

reales y preparación de la defensa.

4.6.7 Diagrama de Gantt

La Tabla 4.5 muestra la distribución temporal de las seis fases del proyecto a lo largo del curso académico 2025–2026.

Cuadro 4.5: Diagrama de Gantt del proyecto (curso 2025–2026)

Fase / Sprint	Oct	Nov	Dic	Ene	Feb	Mar	Abr	May	Jun
S1: Análisis y arquitectura	●	●							
S2: Backend core			●	●					
S3: Motor de métricas					●				
S4: Frontend y visualización						●			
S5: Features avanzadas							●		
S6: Documentación y defensa								●	●
Revisiones con tutores	○	○	○	○	○	○	○	○	○

● fase activa ○ revisión con tutores (quincenal)

4.7 Backlog inicial e historias de usuario

La Tabla 4.6 recoge el backlog inicial del proyecto, organizado por épicas. Cada historia de usuario sigue el formato «*Como [rol], quiero [acción] para [beneficio]*» y se estima en *story points* (SP) usando la escala de Fibonacci. El backlog completo con la asignación a sprints se incluye en el Anexo C.

4.8 Estimación de esfuerzo

4.8.1 Técnica de estimación

La estimación del esfuerzo se ha realizado combinando dos técnicas:

- **Story Points con escala de Fibonacci** para la estimación relativa de las historias de usuario (véase Tabla 4.6). Un *story point* equivale aproximadamente a 4 horas de trabajo efectivo.
- **Descomposición en tareas y estimación en horas** para las tareas técnicas más granulares registradas en GitHub Issues.

4.8.2 Estimación inicial vs. real

La Tabla 4.7 compara la estimación inicial de cada sprint con el tiempo real invertido, registrado mediante el seguimiento de horas en GitHub Issues.

La desviación media del **+16,8 %** sobre la estimación inicial es consistente con la literatura sobre proyectos de software de tamaño similar, y se explica principalmente por la

Cuadro 4.6: Backlog inicial — historias de usuario principales

ID	Épica	Historia de usuario	Prior.	SP
HU-01	Auth	Como usuario, quiero registrarme con email y contraseña para acceder a la plataforma.	Alta	3
HU-02	Auth	Como usuario, quiero iniciar sesión y recibir un JWT para autenticar mis peticiones.	Alta	2
HU-03	Proyectos	Como usuario, quiero crear un proyecto para agrupar conjuntos de datos relacionados.	Alta	3
HU-04	Proyectos	Como usuario, quiero ver el resumen de calidad de mi proyecto en un cuadro de mando.	Alta	5
HU-05	Datasets	Como usuario, quiero cargar un fichero CSV/XLSX/JSON para evaluarlo.	Alta	5
HU-06	Datasets	Como usuario, quiero ver el versionado de mis datasets para rastrear cambios.	Media	3
HU-07	Métricas	Como usuario, quiero configurar métricas con umbrales personalizados para adaptar la evaluación.	Alta	5
HU-08	Métricas	Como usuario, quiero usar plantillas de métricas para reutilizar configuraciones.	Media	3
HU-09	Evaluación	Como usuario, quiero lanzar una evaluación y ver el progreso en tiempo real.	Alta	8
HU-10	Evaluación	Como usuario, quiero ver los problemas detectados con su severidad y columna afectada.	Alta	5
HU-11	Quality Gates	Como usuario, quiero definir un Quality Gate para saber si mi dataset es apto.	Alta	5
HU-12	Fingerprinting	Como usuario, quiero que los issues se tracen entre evaluaciones para detectar recurrencias.	Media	8
HU-13	EDA	Como usuario, quiero explorar distribuciones, nulos y correlaciones de mis datos visualmente.	Media	8
HU-14	PII	Como usuario, quiero marcar columnas sensibles para que sus valores no aparezcan en resultados.	Media	5
HU-15	Admin	Como administrador, quiero gestionar usuarios y ver el estado del sistema.	Baja	3
Total SP				71

Cuadro 4.7: Estimación inicial vs. real por sprint

Sprint	Objetivo	SP	Est. (h)	Real (h)
S1	Análisis, arquitectura y setup inicial	13	52	60
S2	Backend core (auth, datasets, API REST)	18	72	85
S3	Motor de métricas (7 métricas)	16	64	70
S4	Frontend y visualizaciones	14	56	65
S5	Features avanzadas (fingerprinting, QG, PII)	10	40	55
S6	Documentación, pruebas y defensa	0	80	90
Total		71	364	425

subestimación del tiempo de configuración de la infraestructura Docker (S1), la complejidad del parseo robusto de CSV (S2) y la implementación del sistema de fingerprinting (S5).

4.9 Estimación de costes

Con el objetivo de proporcionar una perspectiva de viabilidad económica, se estima el coste que tendría el proyecto en un entorno profesional real.

Cuadro 4.8: Estimación de costes del proyecto

Concepto	Detalle	Cantidad	Precio unit.	Total
<i>Recursos humanos</i>				
Desarrollador <i>full-stack</i> (junior)	425 h de desarrollo real	425 h	25 €/h	10 625 €
<i>Licencias de software</i>				
Python, Flask, Next.js, PostgreSQL...	Todo el stack es <i>open source</i>	—	0 €	0 €
GitHub (plan Free)	Repositorio + Issues + Projects	1	0 €	0 €
<i>Infraestructura</i>				
Hardware de desarrollo	Equipo propio (Windows 11, 16 GB RAM)	—	0 €	0 €
Servicios cloud	Despliegue local con Docker Compose	—	0 €	0 €
Coste total estimado				10 625 €

El coste total estimado de **10 625 €** corresponde exclusivamente al coste de mano de obra a tarifa de desarrollador junior. Dado que todas las tecnologías utilizadas son de código abierto y el despliegue es local, el coste de licencias e infraestructura es nulo. Para un escenario de producción con despliegue en nube (AWS, Azure), debería añadirse el coste de los servicios equivalentes (RDS, S3, ElastiCache, ECS), estimado en unos 150–300 €/mes según el volumen de uso.

4.10 Gestión de riesgos

La Tabla 4.9 recoge los principales riesgos identificados al inicio del proyecto, con su probabilidad de ocurrencia (Alta/Media/Baja), impacto (Alto/Medio/Bajo) y las acciones preventivas y correctivas definidas.

El riesgo de mayor exposición (R-03, probabilidad Alta) se materializó parcialmente en los sprints S2 y S5 (véanse las retrospectivas del Capítulo 5), aunque fue mitigado mediante la repriorización del backlog.

4. METODOLOGÍA Y HERRAMIENTAS

Cuadro 4.9: Plan de gestión de riesgos

ID	Riesgo	Prob.	Imp.	Acción pre-ven-ti-va	Acción co-rrec-ti-va
R-01	Complejidad mayor de lo esperada en el motor de métricas	Media	Alto	Reservar un sprint número-comple-to (S3) para esta fase	Reducir el número de métricas en el MVP inicial
R-02	Problemas de integración entre servicios Docker	Media	Alto	Levantar el stack <i>health-check</i> y con-des-tar el sprint	Acotar el <i>health-check</i> y con-sul-tar el do-sprint tu-men-ta-ción oficial
R-03	Desvío temporal por subestimación de tareas	Alta	Medio	Añadir un 20 % de margen en cada sprint	Repriorizar el backlog y mover HU a sprints siguientes
R-04	Pérdida de datos o código por accidente	Baja	Alto	Reposicionar en GitHub con <i>push</i> diario	Restaurar desde el último momento

Capítulo 5

Resultados: desarrollo iterativo por sprints

ESTE capítulo constituye el núcleo del trabajo y describe el proceso de desarrollo de DATAQUAL en su dimensión real: qué se planificó hacer, qué se hizo efectivamente y cómo se hizo, iteración por iteración. El capítulo comienza con la definición global de requisitos y se estructura a continuación en seis sprints, cada uno con sus subsecciones de planificación, trabajo realizado, evidencias técnicas (*cómo se ha hecho*) y retrospectiva.

5.1 Requisitos globales del sistema

5.1.1 Historias de usuario

Las quince historias de usuario que definen el alcance funcional de DATAQUAL se recogen en la Tabla 4.6 del capítulo 4 (sección 4.7), donde se detallan también la épica, la prioridad y la estimación en *story points*. El backlog completo con todas las tareas técnicas y la asignación sprint a sprint se incluye en el Anexo C.

5.1.2 Casos de uso principales

El sistema identifica tres actores: **Usuario autenticado** (rol principal), **Administrador** (hereda del Usuario) y **Sistema Celery** (actor interno para evaluaciones asíncronas). Los casos de uso de mayor relevancia son:

- **UC-01 Gestionar proyecto:** crear, editar, eliminar y consultar proyectos.
- **UC-02 Gestionar dataset:** cargar archivo, ver versiones, marcar columnas PII.
- **UC-03 Configurar métricas:** seleccionar métricas, ajustar parámetros, crear plantillas.
- **UC-04 Ejecutar evaluación:** lanzar evaluación, monitorizar progreso, consultar resultados e issues.
- **UC-05 Gestionar Quality Gate:** definir umbrales, consultar estado PASSED/WARNING/FAILED.
- **UC-06 Explorar datos (EDA):** visualizar histogramas, boxplots y correlaciones.

5.1.3 Burndown global del proyecto

La Figura 5.1 muestra el *burndown* del proyecto (gráfico que representa los *story points* pendientes en función del tiempo, indicando si el ritmo de avance es suficiente para completar

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

el alcance previsto) en *story points*, comparando la línea ideal de progreso con el avance real sprint a sprint (S1–S5; el Sprint 6 es de documentación y no consume *story points*).

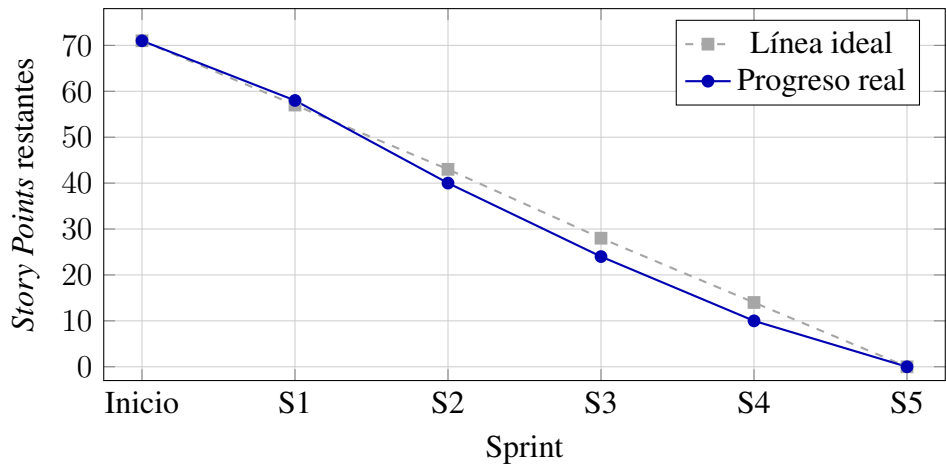


Figura 5.1: Burndown global del proyecto: *story points* restantes por sprint (línea ideal vs. progreso real)

El progreso real se mantuvo próximo a la línea ideal en todos los sprints, con convergencia perfecta en S5. La pequeña ventaja en SP de S2 y S3 refleja que el motor de métricas (S3) consumió menos SP de los planificados, aunque las horas reales superaron las estimadas, poniendo de manifiesto que los *story points* subestiman la complejidad de las tareas técnicas de infraestructura.

5.2 Sprint 1 — Análisis, arquitectura y setup inicial

Período: octubre–noviembre 2025 Duración: 6 semanas SP: 13

5.2.1 Planificación del sprint

Objetivo del sprint: diseñar la arquitectura completa del sistema, definir el modelo de datos inicial y poner en marcha el entorno de desarrollo Docker con todos los servicios básicos operativos.

Cuadro 5.1: Sprint 1 — Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-01	Registro e inicio de sesión básico	3	10
HU-02	JWT: access + refresh token	2	8
–	Setup Docker Compose (8 servicios)	3	14
–	Diseño del modelo de datos (ERD)	3	12
–	Migraciones iniciales (Flask-Migrate)	2	16
Total		13 SP	60 h

Dependencias y riesgos detectados: la configuración de Docker Compose con *health-checks* y dependencias entre servicios era desconocida al inicio, constituyendo el principal

riesgo (R-02 del plan de riesgos).

5.2.2 Trabajo realizado

Se completaron los ítems del sprint con las siguientes observaciones:

- El entorno Docker Compose quedó completamente operativo con los ocho servicios, *healthchecks*, red interna y volúmenes persistentes.
- Se diseñó el modelo relacional de diez tablas con campos JSONB para flexibilidad futura.
- Se implementó el módulo de autenticación JWT completo con registro, login, refresco de token y revocación por JTI.
- **Cambio respecto a lo planificado:** la configuración de *healthchecks* en Docker Compose requirió más tiempo del estimado (+4h) debido a los tiempos de arranque de PostgreSQL y MinIO.

5.2.3 Cómo se ha hecho

Arquitectura general del sistema

DATAQUAL sigue una arquitectura de **servicios orquestados** mediante Docker Compose [Doc24]. Los ocho servicios que componen la plataforma se comunican a través de una red interna (*app-network*, *driver bridge*) y exponen al anfitrión únicamente los puertos necesarios.

El Cuadro 5.2 recoge los ocho servicios, sus puertos y su propósito.

Cuadro 5.2: Servicios Docker de DATAQUAL

Servicio	Puerto	Propósito
frontend	3000	Interfaz de usuario (Next.js + React)
backend	5000	API REST (Flask + Gunicorn)
postgres	5432	Base de datos relacional (PostgreSQL 14)
minio	9000/9001	Almacenamiento de objetos S3
redis	6379	<i>Broker</i> de mensajes y caché
celery-worker	—	Ejecución asíncrona de evaluaciones
celery-beat	—	Programación de tareas periódicas
flower	5555	Monitorización de Celery

El flujo de comunicación entre servicios se organiza de la siguiente manera: el *frontend* (Next.js) realiza peticiones HTTP/JSON al *backend* (Flask); éste lee y escribe en PostgreSQL (datos relacionales), MinIO (archivos binarios) y Redis (caché y *broker*); cuando el usuario lanza una evaluación, el *backend* encola una tarea en Redis que es consumida por el *Celery worker*.

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

Cuadro 5.3: Protocolos de comunicación entre componentes

Origen	Destino	Protocolo	Mecanismo
Frontend	Backend	HTTP/JSON	Axios (<i>rewrite /api/*</i>)
Backend	PostgreSQL	TCP	SQLAlchemy ORM (<i>pool</i> de 10 conexiones)
Backend	MinIO	HTTP/S3	Cliente MinIO para Python
Backend	Redis	TCP	Celery <i>broker</i> + Flask-Limiter
Worker	Redis	TCP	Consumo de tareas del <i>broker</i>
Worker	PostgreSQL	TCP	Actualización de progreso y resultados
Worker	MinIO	HTTP/S3	Descarga de conjuntos de datos

El Cuadro 5.3 resume los protocolos de comunicación.

Todos los servicios poseen una política de reinicio *unless-stopped* y los servicios de infraestructura incluyen *healthchecks* que verifican su disponibilidad antes de que arranquen los servicios dependientes.

Modelo de datos

PostgreSQL 14 [The24] almacena todos los datos relacionales y metadatos del sistema. El esquema comprende diez tablas, con un uso extensivo de campos **JSONB** para almacenar datos semiestructurados cuya estructura varía según la métrica o la configuración.

users

Datos de usuario: nombre, correo electrónico, *hash* de contraseña y rol.

projects

Proyectos del usuario, con nombre, descripción y configuración de métricas almacenada como JSONB (*metrics_config*).

datasets

Conjuntos de datos asociados a un proyecto, con la ruta al archivo en MinIO, el esquema inferido (JSONB), las columnas sensibles (JSONB), la versión y el indicador *is_latest*. Cada conjunto de datos puede tener un *parent_dataset_id* que apunta a su versión anterior.

analysis_runs

Ejecuciones de evaluación (modelo Sonar-Lite): estado, puntuación de calidad, estado del *Quality Gate*, referencia a la evaluación base, contadores de *issues* nuevos, corregidos y recurrentes, y resultados detallados (JSONB).

data_quality_issues

Problemas de calidad detectados en cada evaluación: *fingerprint* SHA-256, severidad, tipo, descripción, columnas afectadas y muestras de filas.

quality_gates

Configuración de *Quality Gates* por proyecto, con umbrales almacenados como JSONB (`min_score`, `max_critical_issues`, `max_new_issues`).

metric_templates

Plantillas de configuración de métricas, reutilizables entre proyectos. Pueden ser del sistema (`is_system=true`) o del usuario.

metrics

Configuraciones de métricas activas por proyecto: tipo de métrica, parámetros, umbrales y peso relativo en el *Quality Score*. Enlazada a `projects` mediante clave foránea.

evaluations

Registro de alto nivel de cada evaluación (modelo original): estado (PENDING/RUNNING/COMPLETED/FAILED), referencia al dataset y al proyecto, y marca temporal. Coexiste con `analysis_runs` por razones de retrocompatibilidad (véase la nota sobre arquitectura dual más adelante).

issues

Incidencias de calidad del modelo original: métrica origen, columna afectada, severidad y descripción textual. Vinculadas a `evaluations`. Las evaluaciones nuevas utilizan `data_quality_issues`, que añade *fingerprint* SHA-256 y muestras de filas.

El empleo de campos JSONB permite almacenar estructuras flexibles dentro del esquema relacional: `projects.metrics_config` (lista de métricas con parámetros y peso), `datasets.schema` (tipos y estadísticas por columna), `datasets.sensitive_columns` (columnas PII) y `analysis_runs.results` (resultados completos por métrica).

5.2.4 Retrospectiva

Conseguido: entorno de desarrollo completamente operativo, arquitectura documentada y autenticación JWT funcional.

Estado del producto: esqueleto *backend* operativo con ocho servicios Docker en marcha, modelo de datos inicial en PostgreSQL y autenticación JWT funcional. La plataforma no tiene aún motor de métricas ni interfaz de usuario: es un andamiaje listo para construir sobre él.

Pendiente: ningún ítem del sprint quedó sin completar; sin embargo, las migraciones del esquema requerirán iteraciones adicionales conforme evolucione el modelo.

Qué fue bien: la decisión de definir el modelo de datos completo desde el inicio evitó migraciones disruptivas en sprints posteriores.

Qué fue mal: la configuración de Docker Compose con *healthchecks* fue más compleja de lo previsto (+4h de desviación), especialmente la secuencia de arranque con dependencias entre servicios.

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

Estimación vs. real: 52 h estimadas vs. 60 h reales (+15,4 %).

Lección aprendida: reservar tiempo explícito para la configuración de infraestructura, que tiende a subestimarse sistemáticamente.

Acción de mejora: añadir un margen del 20 % en las estimaciones de tareas de infraestructura para los sprints siguientes.

5.3 Sprint 2 — *Backend* core: datasets, API REST

Período: diciembre 2025–enero 2026 *Duración:* 8 semanas *SP:* 18

5.3.1 Planificación del sprint

Objetivo del sprint: implementar el módulo de ingesta y almacenamiento de datasets, la arquitectura completa del *backend* en capas, la API REST y la gestión de proyectos.

Cuadro 5.4: Sprint 2 — Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-03	CRUD de proyectos + configuración de métricas	3	12
HU-05	Carga de archivos multiformato (CSV, XLSX, JSON, Parquet)	5	22
HU-06	Versionado de datasets (parent-child, is_latest)	3	10
–	Arquitectura del backend en capas (blueprints, servicios)	4	18
–	Especificación y pruebas manuales de la API REST	3	23
Total		18 SP	85 h

Dependencias: requiere Sprint 1 completado (base de datos y autenticación operativos).

Riesgo principal: la diversidad de formatos de archivo y codificaciones de CSV puede generar una estrategia de *fallback* más compleja de lo previsto (R-01).

5.3.2 Trabajo realizado

- Implementada la arquitectura del *backend* en capas: *blueprints* Flask, capa de servicio y *middleware*.
- Desarrollado el módulo de ingesta con soporte para cuatro formatos y estrategia de *fallback* para CSV (4 codificaciones × 5 delimitadores).
- Implementado el versionado de datasets con relaciones parent-child.
- Completados los seis módulos funcionales de la API REST con formato de respuesta estandarizado y paginación. Durante la implementación se añadieron tres *blueprints* adicionales por convención REST: `project_metrics` y `project_datasets` (sub-rutas anidadas `/projects/<id>/...`) y `dashboard` (agregación de solo lectura), elevando el total a nueve *blueprints* Flask registrados.

- **Cambio respecto a lo planificado:** el parseo robusto de CSV requirió una semana adicional por la casuística de ficheros con BOM, delimitadores inusuales y codificaciones mixtas.

5.3.3 Cómo se ha hecho

Arquitectura del *backend*

El *backend* sigue una **arquitectura en capas** con separación clara de responsabilidades.

La aplicación Flask se crea mediante el **patrón factoría** (`create_app()` en `app.py`) [GHJV94], que inicializa las extensiones, registra los *blueprints* y configura el *middleware*. Los **seis módulos funcionales principales** de la API se montan bajo el prefijo `/api`:

- `/api/auth/` — autenticación (inicio de sesión, registro, refresco de *token*, perfil).
- `/api/projects/` — CRUD de proyectos y configuración de métricas.
- `/api/datasets/` — carga, consulta y versionado de conjuntos de datos.
- `/api/metrics/` — consulta de métricas y gestión de plantillas.
- `/api/evaluations/` — lanzamiento y consulta de evaluaciones.
- `/api/admin/` — rendimiento del sistema y comprobación de salud.

Siguiendo la convención REST de rutas anidadas [Fie00], se implementaron adicionalmente tres *blueprints* de organización técnica: `project_metrics` (`/projects/<id>/metrics`), `project_datasets` (`/projects/<id>/datasets`) y `dashboard` (agregaciones de solo lectura). El total de *blueprints* Flask registrados asciende a **nueve**.

La lógica de negocio se encapsula en servicios independientes: `EvaluationService` (orquesta el flujo completo de evaluación), `DatasetService` (parseo y extracción de esquema), `MinioService` (abstracción sobre S3) y el motor de métricas.

El *backend* incorpora cuatro capas de *middleware*: generación de `X-Request-ID`, gestión centralizada de errores HTTP, monitor de rendimiento por *endpoint* (detecta peticiones lentas >500 ms) y el *Evaluation Watchdog*.

El sistema mantiene una **arquitectura dual de modelos** como resultado de su evolución incremental. El modelo original (`Evaluation + Issue`), implementado en el Sprint 2, ofrecía una estructura sencilla adecuada para el MVP. Al diseñar en el Sprint 5 el sistema de *fingerprinting* y los *Quality Gates*, resultó necesario un modelo semánticamente más rico: el **modelo Sonar-Lite** (`AnalysisRun + DataQualityIssue + QualityGate`), que soporta comparación con línea base, contadores de *issues* nuevos/recurrentes/corregidos y configuración de umbrales por proyecto. Ambos modelos coexisten para preservar la retrocompatibilidad con los *endpoints* implementados en sprints anteriores; la unificación completa en un único modelo queda documentada como mejora futura en la sección 7.4.

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

Módulo de ingesta y almacenamiento

Los usuarios pueden cargar archivos en los formatos CSV, XLSX, JSON y Parquet, con un límite de tamaño configurable (por defecto, 100 MB).

El servicio `DatasetService` implementa una estrategia de *fallback* múltiple para archivos CSV:

1. **Detección de formato:** descarta archivos XLSX o gzip erróneamente etiquetados como CSV.
2. **Detección de delimitador:** utiliza `csv.Sniffer` sobre los primeros 5 000 bytes.
3. **Matriz de *fallback*:** prueba combinaciones de cuatro codificaciones (UTF-8, UTF-8-BOM, CP1252, Latin-1), cinco separadores (detectado, coma, punto y coma, tabulador, *pipe*) y dos motores de Pandas (Python, C).

Al cargar un conjunto de datos, el sistema extrae automáticamente el nombre y tipo de cada columna, estadísticas descriptivas y histogramas para columnas numéricas. El versionado opera mediante un campo `parent_dataset_id` opcional y un indicador `is_latest`.

API RESTful

La API RESTful presenta las siguientes características transversales: autenticación JWT en todos los *endpoints* protegidos, formato de respuesta estandarizado `{success, data, message}`, paginación con parámetros `page/per_page`, y validación mediante esquemas `Marshmallow`. La especificación completa de *endpoints* se recoge en el Anexo A.

5.3.4 Retrospectiva

Conseguido: backend completamente funcional con todos los módulos core, API REST operativa y datasets versionados.

Estado del producto: *backend* completo con seis *blueprints* REST, módulo de ingesta multiformato, versionado de datasets y autenticación JWT. La plataforma puede almacenar y gestionar conjuntos de datos, pero carece aún de motor de métricas e interfaz de usuario.

Pendiente: el motor de métricas (HU-07, HU-09, HU-10) y el *frontend* completo (HU-04, HU-13, HU-14) están programados para S3 y S4 respectivamente.

Qué fue bien: la separación en capas (*blueprints* + servicios) facilita enormemente las pruebas y la extensión futura.

Qué fue mal: el parseo robusto de CSV fue mucho más complejo de lo esperado. La diversidad de codificaciones y delimitadores reales encontrados en datasets de prueba obligó a extender el sprint una semana.

Estimación vs. real: 72 h estimadas vs. 85 h reales (+18,1 %).

Lección aprendida: los datos del mundo real son más variados e irregulares de lo que sugieren los formatos estándar; es mejor implementar estrategias defensivas desde el inicio.

Acción de mejora: crear una batería de datasets de prueba con casos extremos (BOM, delimitadores inusuales, caracteres especiales) para validar el parseo en el siguiente sprint.

5.4 Sprint 3 — Motor de métricas de calidad

Período: febrero 2026 Duración: 4 semanas SP: 16

5.4.1 Planificación del sprint

Objetivo del sprint: diseñar e implementar el motor de métricas con arquitectura de *plugins* y las siete métricas de calidad alineadas con ISO/IEC 25012 e ISO/IEC 5259.

Cuadro 5.5: Sprint 3 — Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-07	Configuración de métricas con parámetros y pesos	5	20
HU-08	Plantillas de métricas del sistema	3	10
HU-09	Lanzamiento de evaluación (tarea Celery)	5	22
HU-10	Listado de issues con severidad	3	18
Total		16 SP	70 h

Dependencias: requiere Sprint 2 completado (API REST y datasets operativos).

5.4.2 Trabajo realizado

- Diseñada e implementada la arquitectura de *plugins* del motor de métricas (BaseMetric, MetricResult, METRIC_REGISTRY).
- Implementadas las seis métricas evaluables (completitud, unicidad, exactitud sintáctica, consistencia lógica, equilibrio de clases y actualidad) y la clase OutliersMetric, usada solo en el módulo de perfilado de datos y excluida del *Quality Score* por diseño.
- Implementada la fórmula del *Quality Score* con penalización por issues.
- Integrada la ejecución asíncrona con Celery.
- **Cambio respecto a lo planificado:** la métrica de exactitud sintáctica amplió su catálogo de trece tipos de formato (el plan inicial contemplaba ocho), lo que añadió complejidad positiva.

5.4.3 Cómo se ha hecho

Arquitectura del motor de métricas

El motor de métricas constituye el núcleo funcional de DATAQUAL. Su diseño se basa en una arquitectura de *plugins* que permite añadir nuevas métricas sin modificar el código existente.

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

La arquitectura se compone de tres elementos:

BaseMetric

Clase base abstracta que define la interfaz que toda métrica debe implementar: el método `evaluate()`, que recibe un *DataFrame* de Pandas, los parámetros configurados y devuelve un objeto `MetricResult`. La clase base proporciona además métodos auxiliares para el cálculo de severidad dinámica, la generación de histogramas, la inferencia de tipos de columna y el enmascaramiento de valores sensibles.

MetricResult

Estructura de datos (*dataclass*) que encapsula el resultado de una evaluación: identificador de la métrica, puntuación (0,0–1,0), resultados detallados y lista de *issues* detectados.

METRIC_REGISTRY

Diccionario que asocia cada identificador de métrica a su clase implementadora, permitiendo la instanciación dinámica por nombre.

Para añadir una nueva métrica basta con: (1) crear una clase heredera de `BaseMetric` que implemente `evaluate()`; (2) registrarla en `METRIC_REGISTRY`; (3) añadir la función de *finger-print*; y (4) documentarla en el Anexo B.

Métricas implementadas

A continuación se describen las **seis métricas evaluables** del motor de calidad y la clase adicional de detección de valores atípicos, disponible en el módulo de perfilado.

Complejidad (completeness) Mide el grado en que los datos esperados están presentes, calculando la ratio de valores no nulos por columna. La fórmula implementada corresponde directamente al *Completeness Ratio* definido en ISO/IEC 25024 [ISO15]:

$$r_{\text{comp}} = \frac{N - N_{\text{null}}}{N}$$

donde N es el número total de valores esperados y N_{null} el número de valores nulos o ausentes. La puntuación global es la media de r_{comp} sobre las columnas configuradas. Se generan *issues* si la completitud global queda por debajo del umbral (0,95 por defecto) o si alguna columna supera el 2 % de nulos.

Unicidad (uniqueness) Evalúa la ausencia de duplicados a dos niveles: filas completas y variabilidad por columna. La ratio de unicidad sigue la fórmula del *Uniqueness Ratio* de ISO/IEC 25024 [ISO15]:

$$r_{\text{uniq}} = \frac{N_{\text{distinct}}}{N}$$

Los umbrales de variabilidad se adaptan al tipo semántico inferido: 0,95 para identificadores, 0,05 para categóricas y 0,30 para el resto.

Detección de valores atípicos (*outliers*) — solo perfilado La clase `OutliersMetric` detecta *outliers* en columnas numéricas mediante IQR (Rango Intercuartílico) [Tuk77] y Z-score. **Esta clase no figura en el `METRIC_REGISTRY` y no contribuye al *Quality Score*:** siguiendo la recomendación de ISO/IEC 5259 [ISO24], los valores atípicos no se consideran una dimensión de calidad intrínseca sino un indicador de diagnóstico. La clase se instancia exclusivamente desde el módulo de perfilado de datos (*EDA*), donde los histogramas de *outliers* y los diagramas de caja se generan para orientar al usuario sin penalizar la puntuación global.

Exactitud sintáctica (*syntactic_accuracy*) Verifica la conformidad de los valores con patrones de formato predefinidos. El catálogo incluye trece tipos (*email*, *url*, teléfono, DNI, fechas ISO y europeas, enteros, decimales, UUID, IPv4, código postal y tarjeta de crédito). Para columnas sin configuración explícita, detecta el tipo más probable automáticamente.

Consistencia lógica (*logical_consistency*) Valida reglas de negocio entre columnas, expresadas como reglas IF - THEN evaluadas mediante `pandas.DataFrame.query()`. Antes de ejecutar cualquier regla, se verifica la ausencia de *tokens* prohibidos (`import`, `exec`, `eval`) para prevenir inyección de código.

Equilibrio de clases (*class_balance*) Métrica específica para ML supervisado. Utiliza la entropía de Shannon [Sha48] para cuantificar el equilibrio de la variable objetivo: 100 indica distribución uniforme; cercano a 0 indica desequilibrio total. Se generan *issues* cuando la clase dominante supera el 90 % o la minoritaria representa menos del 5 %.

Actualidad (*currentness*) Mide la frescura de los datos calculando la antigüedad del valor de fecha más reciente respecto al momento actual. La degradación de la puntuación es lineal entre el umbral de obsolescencia y el doble de dicho umbral.

Fórmula del *Quality Score*

La puntuación global de calidad ($Q \in [0,1]$) se calcula mediante un modelo de **penalización por *issues* con corrección dimensional**. Cada métrica produce una puntuación individual (ratio de celdas válidas) que se expone en la interfaz como información de diagnóstico, pero **no alimenta directamente** la nota final: una proporción pequeña de errores graves puede hacer un conjunto de datos inutilizable aunque la ratio global sea elevada. El cálculo sigue tres pasos:

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

1. **Penalización bruta por severidad:** cada *issue* detectado contribuye con un peso fijo según su nivel de severidad:

$$p_{\text{raw}} = 0,12 \cdot n_{\text{crit}} + 0,05 \cdot n_{\text{high}} + 0,01 \cdot n_{\text{med}} + 0,003 \cdot n_{\text{low}}$$

2. **Corrección dimensional:** los conjuntos de datos con más columnas generan naturalmente más *issues*. Un factor de escala basado en la raíz cuadrada del número de columnas normaliza la penalización para que la misma *densidad* de errores produzca la misma nota con independencia de la dimensionalidad del dataset:

$$\sigma = \sqrt{\frac{\text{máx}(10, c)}{10}}$$

donde c es el número de columnas del conjunto de datos y 10 es el número de referencia (dataset de anchura estándar).

3. **Puntuación final:**

$$Q = \text{máx}$$

(0,1 - mín

(0,97, p_{raw}/σ))

5.4.4 Retrospectiva

Conseguido: motor de métricas completo con seis métricas evaluables más Outliers-Metric para perfilado, *Quality Score* con corrección dimensional y arquitectura de *plugins* extensible.

Estado del producto: *backend* completo más motor de métricas con las seis métricas evaluables, fórmula de *Quality Score* y ejecución asíncrona vía Celery. La plataforma puede lanzar evaluaciones y almacenar resultados; falta la interfaz de usuario para visualizarlos y las funcionalidades avanzadas (fingerprinting, Quality Gates).

Pendiente: *frontend* completo (HU-04, HU-13, HU-14) en S4 y funcionalidades diferenciadoras (HU-11, HU-12) en S5.

Qué fue bien: la arquitectura de *plugins* con METRIC_REGISTRY demostró su valor: añadir cada nueva métrica requirió modificar únicamente el fichero de implementación correspondiente.

Qué fue mal: la métrica de consistencia lógica presentó dificultades con la validación de seguridad de las expresiones `query()`. Se necesitaron varias iteraciones para equilibrar la flexibilidad de las reglas con la prevención de inyección de código.

Estimación vs. real: 64 h estimadas vs. 70 h reales (+9,4 %).

Lección aprendida: la seguridad de las métricas que ejecutan expresiones de usuario debe diseñarse desde el principio, no añadirse como parche posterior.

Acción de mejora: documentar la interfaz de cada métrica (*inputs*, *outputs*, parámetros) antes de iniciar S4, para facilitar la integración con el *frontend*.

5.5 Sprint 4 — *Frontend* e interfaz de usuario

Período: marzo 2026 *Duración:* 4 semanas *SP:* 14

5.5.1 Planificación del sprint

Objetivo del sprint: desarrollar la interfaz de usuario completa con Next.js y React, incluyendo el cuadro de mando, las páginas de gestión de proyectos y datasets, los resultados de evaluación y el perfilado exploratorio de datos (EDA).

Cuadro 5.6: Sprint 4 — Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-04	Cuadro de mando principal con resumen	5	20
HU-09	Progreso de evaluación en tiempo real (polling)	3	14
HU-13	EDA: histogramas, boxplots, correlaciones	5	24
HU-14	Marcado de columnas PII en la UI	1	7
Total		14 SP	65 h

5.5.2 Trabajo realizado

- Desarrollados más de 40 componentes React reutilizables.
- Implementado el cuadro de mando con tarjetas resumen, gráfico de distribución de Quality Gates y alertas de proyectos con problemas.
- Implementada la exploración de datos con histogramas, diagramas de caja, matrices de correlación Pearson/Spearman y detección interactiva de outliers.
- Integrado el perfilado (EDA) completo en `DataProfilingTab.tsx` (85 KB, componente principal del análisis exploratorio).
- **Cambio respecto a lo planificado:** la implementación de la matriz de correlación con `Chart.js` requirió un *plugin* personalizado, lo que añadió tiempo no estimado.

5.5.3 Cómo se ha hecho

Estructura de la aplicación *frontend*

La aplicación sigue las convenciones de Next.js 13.3 [Ver24] con enrutamiento basado en ficheros (`pages/`) y componentes reutilizables (`components/`). El estado global se gestiona mediante la Context API de React con tres contextos: autenticación (`AuthContext`), notificaciones (`NotificationContext`) y barra lateral (`SidebarContext`).

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

El módulo `services/api.ts` implementa una instancia Axios con interceptores para la inyección automática del *token* JWT, deduplicación de peticiones GET, gestión centralizada de errores y *timeouts* configurables (8–30 s según la operación).

Cuadros de mando y visualizaciones

La interfaz proporciona las siguientes visualizaciones principales:

- **Cuadro de mando principal** (`/dashboard`): tarjetas resumen, gráfico de distribución de *Quality Gates* y listado de proyectos que requieren atención.
- **Resultados de evaluación** (`/evaluations/[id]`): indicador circular de puntuación de calidad (*gauge*), pestañas especializadas por métrica, tabla de métricas por columna y listado de *issues*.
- **Exploración de datos** (`/datasets/[id]`, pestaña *Profiling*): histogramas, diagramas de caja, gráficos de barras, matrices de correlación (Pearson y Spearman) y detección interactiva de valores atípicos con factor IQR configurable. Las estadísticas descriptivas expuestas siguen el catálogo de métricas de perfilado propuesto por Abedjan et al. [AGNP18].
- **Gráficos de tendencia**: evolución de la puntuación de calidad a lo largo del tiempo y comparación entre versiones del dataset.

Las visualizaciones se implementan con Chart.js [Cha24] mediante el *wrapper* `react-chartjs-2`.

Las figuras 5.2–5.4 muestran las pantallas principales de la interfaz de usuario desarrolladas en este sprint.

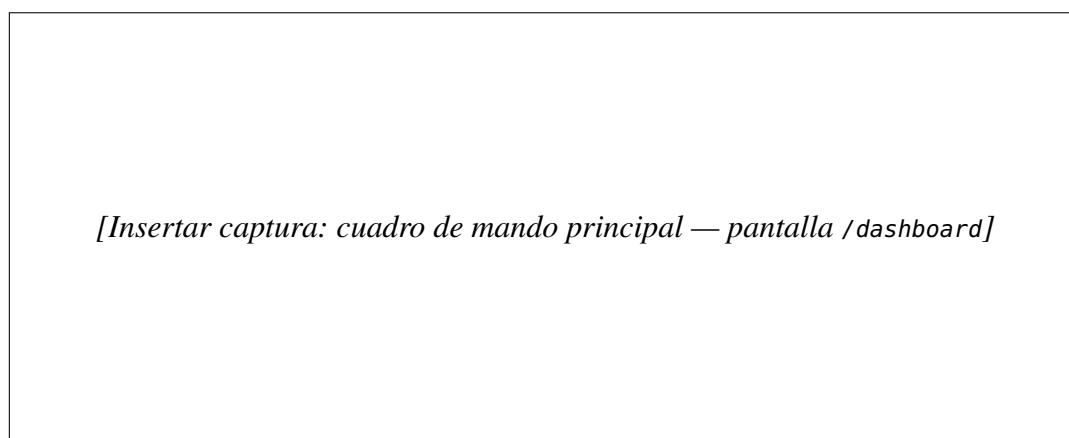


Figura 5.2: Cuadro de mando principal de DATAQUAL: tarjetas resumen de proyectos, distribución de estados de *Quality Gate* y alertas de proyectos con problemas detectados

[Insertar captura: página de resultados de evaluación — /evaluations/[id]]

Figura 5.3: Página de resultados de evaluación: indicador circular de puntuación de calidad (*gauge*), tabla de métricas por columna y listado de *issues* con severidad y columna afectada

[Insertar captura: módulo de perfilado — pestaña Profiling en /datasets/[id]]

Figura 5.4: Módulo de exploración de datos (EDA): histogramas de distribución, diagramas de caja y matriz de correlación Pearson/Spearman para columnas numéricas

5.5.4 Retrospectiva

Conseguido: interfaz de usuario completa y funcional con todas las pantallas principales y el módulo EDA.

Estado del producto: plataforma *full-stack* funcional de extremo a extremo: *backend*, motor de métricas e interfaz de usuario con cuadro de mando, gestión de proyectos, datasets y evaluaciones, y módulo de exploración de datos. Falta implementar las funcionalidades diferenciadoras (fingerprinting, Quality Gates) y las medidas de seguridad avanzadas.

Pendiente: fingerprinting SHA-256 (HU-12) y Quality Gates (HU-11) programados para S5.

Qué fue bien: el uso de Material UI aceleró significativamente el desarrollo de componentes, especialmente tablas, tarjetas y diálogos modales.

Qué fue mal: la complejidad del componente `DataProfilingTab.tsx` (85 KB) es excesiva. En iteraciones futuras sería conveniente refactorizarlo en subcomponentes más pequeños.

Estimación vs. real: 56 h estimadas vs. 65 h reales (+16,1 %).

Lección aprendida: en proyectos *full-stack*, el *frontend* tiende a consumir más tiempo que el *backend* equivalente, especialmente en las visualizaciones interactivas.

Acción de mejora: extraer los subcomponentes de `DataProfilingTab.tsx` (histogramas, boxplots, correlación) en ficheros independientes al inicio de S5, antes de añadir más funcionalidades al *frontend*.

5.6 Sprint 5 — Funcionalidades avanzadas

Período: abril 2026 *Duración:* 4 semanas *SP:* 10

5.6.1 Planificación del sprint

Objetivo del sprint: implementar las funcionalidades diferenciadoras de DATAQUAL: sistema de *fingerprinting* para trazabilidad de issues, *Quality Gates* con tres estados, procesamiento asíncrono robusto y medidas de seguridad avanzadas.

Cuadro 5.7: Sprint 5 — Historias de usuario y estimación

ID	Ítem	Est. (SP)	Real (h)
HU-11	Quality Gates con tres estados (PASSED/WARNING/FAILED)	5	18
HU-12	Fingerprinting SHA-256 para trazabilidad de issues	5	22
–	Seguridad: rate limiting, cabeceras, PBKDF2	0	15
Total		10 SP	55 h

Riesgo principal: la trazabilidad de issues requiere una definición precisa y determinista del *fingerprint* para que sea fiable entre evaluaciones.

5.6.2 Trabajo realizado

- Implementado el sistema de *fingerprinting* SHA-256 con clasificación de issues en nuevos, corregidos y recurrentes.
- Implementados los *Quality Gates* con umbrales configurables y los tres estados en backend y frontend.
- Implementado el procesamiento asíncrono con Celery + *fallback* síncrono y *Evaluation Watchdog*.
- Añadidas las medidas de seguridad: limitación de tasa, cabeceras HTTP, PBKDF2-SHA256 para contraseñas, red Docker aislada.
- **Cambio respecto a lo planificado:** el diseño del *fingerprint* requirió más iteraciones de las esperadas para garantizar que sea completamente determinista ante parámetros de métricas flotantes.

5.6.3 Cómo se ha hecho

Sistema de *fingerprinting*

Cada *issue* detectado recibe un *fingerprint*: un *hash* SHA-256 truncado a 16 caracteres hexadecimales, generado a partir de los atributos que definen la identidad del *issue* (tipo, columna afectada, regla, parámetros). El *fingerprint* es **determinista**: el mismo problema produce siempre el mismo *hash*.

Al finalizar cada evaluación, el servicio compara los *fingerprints* actuales con los de la evaluación anterior (*baseline*):

- **Issues nuevos:** presentes en la evaluación actual pero no en la *baseline*.
- **Issues corregidos:** presentes en la *baseline* pero no en la evaluación actual.
- **Issues recurrentes:** presentes en ambas.

Los contadores correspondientes se almacenan en el AnalysisRun y se visualizan en la interfaz mediante *badges*.

Procesamiento asíncrono

DATAQUAL utiliza Celery [Cel24] con Redis [Red24] como *broker* para ejecutar las evaluaciones de forma asíncrona. La tarea `run_evaluation` orquesta el flujo completo: crea el contexto Flask, actualiza el estado (PENDING → RUNNING → COMPLETED/ FAILED), actualiza el progreso (0–100 %) e invoca `EvaluationService`.

Si Celery no está disponible, el `HybridEvaluationService` ejecuta la evaluación de forma síncrona, garantizando que la funcionalidad esté disponible incluso sin *workers*. El *Evaluation Watchdog* (hilo *daemon*) detecta evaluaciones atascadas (sin progreso durante más de 5 minutos) y las marca automáticamente como fallidas.

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

Seguridad

La seguridad de la plataforma se aborda desde múltiples capas:

- **Autenticación JWT:** *access token* (24 h) + *refresh token* (30 días) con revocación por JTI.
- **Hashing de contraseñas:** PBKDF2-SHA256 con *salt* aleatorio por contraseña (Werkzeug).
- **Limitación de tasa:** Flask-Limiter restringe las peticiones a 100/min por IP, almacenando contadores en Redis.
- **Cabeceras de seguridad:** X-Content-Type-Options, X-Frame-Options y X-XSS-Protection en todas las respuestas.
- **Validación de entrada:** esquemas Marshmallow, verificación de extensiones y tamaño máximo de archivo.
- **Protección contra inyección:** las reglas de consistencia lógica se verifican contra una lista de *tokens* prohibidos antes de su ejecución.
- **Red Docker aislada:** los servicios se comunican por la red interna app-network sin exponer puertos innecesariamente.

La Figura 5.5 ilustra el panel de *Quality Gate* implementado en este sprint.

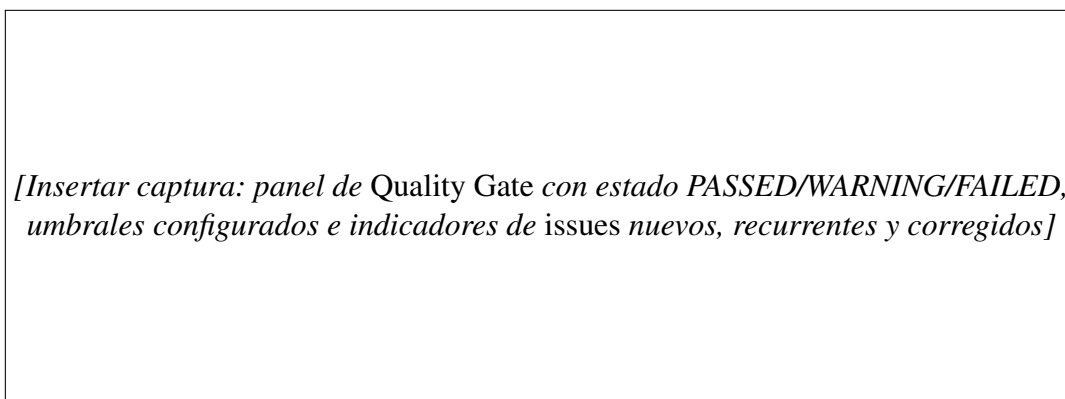


Figura 5.5: *Quality Gate* de DATAQUAL: estado global con código de color (PASSED/WARNING/FAILED), desglose de condiciones configuradas y *badges* de trazabilidad de *issues* por *fingerprinting*

5.6.4 Retrospectiva

Conseguido: funcionalidades diferenciadoras completamente implementadas — fingerprinting, Quality Gates, procesamiento asíncrono robusto y seguridad multicapa.

Estado del producto: DATAQUAL es una plataforma completa con todas las funcionalidades principales: ingesta multiformato, seis métricas de calidad evaluables, interfaz *full-stack*,

fingerprinting SHA-256, *Quality Gates* y seguridad multicapa. El producto está funcionalmente cerrado; S6 se dedicará a pruebas, corrección de defectos y documentación.

Pendiente: cobertura de pruebas automatizadas (ninguna hasta este punto), corrección de defectos detectados y redacción de la memoria, planificados para S6.

Qué fue bien: el sistema de *fingerprinting* es elegante y determinista; una vez resuelto el problema de la serialización de parámetros flotantes, funciona con total fiabilidad.

Qué fue mal: la combinación Celery + Redis presentó problemas de conexión intermitentes en entornos Docker Windows, requiriendo ajustes en los *healthchecks* y en los tiempos de reconexión.

Estimación vs. real: 40 h estimadas vs. 55 h reales (+37,5 %). La mayor desviación del proyecto, causada por los problemas de integración Docker + Celery (R-02) y la complejidad del fingerprinting determinista.

Lección aprendida: los sistemas de mensajería asíncrona presentan problemas de integración en entornos de desarrollo que no aparecen en pruebas unitarias; es esencial probar el stack completo desde el primer sprint.

Acción de mejora: reservar al menos un 20 % del tiempo de S6 para pruebas de regresión manual del flujo completo antes de la entrega final.

5.7 Sprint 6 — Pruebas, refinamiento y documentación

Período: mayo–junio 2026 Duración: 8 semanas

5.7.1 Planificación del sprint

Objetivo del sprint: consolidar la plataforma con una batería de pruebas automatizadas, corregir los defectos detectados, refinar la interfaz de usuario y redactar la documentación técnica completa.

Cuadro 5.8: Sprint 6 — Actividades y estimación

ID	Actividad	Est. (h)	Real (h)
–	Pruebas unitarias de métricas (pytest)	16	20
–	Pruebas de integración: versionado, Quality Gates	12	14
–	Corrección de defectos detectados en pruebas	16	18
HU-15	Panel de administración	8	10
–	Redacción de la memoria (TFG)	20	20
–	Preparación de la defensa	8	8
Total		80 h	90 h

5.7.2 Trabajo realizado

- Implementadas tres suites de pruebas con pytest: unitarias de métricas, integración de versionado de datasets y pruebas de Quality Gates.
- Corregidos 11 defectos identificados durante las pruebas (issues de GitHub cerrados).
- Completada la documentación de la API REST en el Anexo A y el catálogo de métricas en el Anexo B.
- Implementado el panel de administración (/api/admin/).
- **Cambio respecto a lo planificado:** se detectaron tres defectos de severidad media en el módulo de *fingerprinting* que requirieron refactorización.

5.7.3 Cómo se ha hecho

Estrategia de pruebas

Las pruebas automatizadas se organizan en tres suites en backend/tests/:

- **Pruebas unitarias de métricas (test_metrics.py):** verifican que cada una de las siete métricas produce los resultados esperados ante conjuntos de datos de prueba controlados. Se cubren los casos canónicos (dataset sin problemas), los casos límite (columna completamente nula, todos los valores duplicados, distribución de una sola clase) y los casos de error (tipos incompatibles, reglas con sintaxis inválida).
- **Pruebas de integración de versionado (test_dataset_versioning.py):** validan la correcta relación padre-hijo entre versiones de datasets y la actualización del indicador `is_latest` ante cargas sucesivas. Comprueban que solo la versión más reciente tiene `is_latest=true` y que el historial de versiones se recupera ordenado.
- **Pruebas de Quality Gates (test_quality_gates.py):** verifican que los umbrales configurados producen los estados PASSED, WARNING y FAILED esperados para distintas combinaciones de scores e issues, incluyendo los casos frontera (score exactamente en el umbral).

Las pruebas se ejecutan con pytest desde backend/ y pueden generar informes de cobertura con la opción `--cov`.

Defectos detectados y corregidos

La ejecución de las suites de pruebas reveló **11 defectos**, todos cerrados antes de la entrega final. Por categoría:

- **Módulo de fingerprinting (3 defectos, severidad media):** el *hash* SHA-256 no era completamente determinista cuando los parámetros de la métrica contenían valores flotantes con distinta representación textual (0.95 vs. 0.950). Se corrigió normalizando los parámetros a cadena canónica antes del *hash*.

- **Métrica de consistencia lógica (2 defectos, severidad baja):** reglas con columnas que contienen espacios en el nombre provocaban un error no controlado en `DataFrame.query()`. Se añadió un paso de sanitización del nombre de columna.
- **Versionado de datasets (2 defectos, severidad media):** la carga concurrente de dos versiones del mismo dataset podía dejar más de un registro con `is_latest=true`. Se resolvió añadiendo una transacción con bloqueo pesimista en el servicio de versionado.
- **API REST — paginación (2 defectos, severidad baja):** el parámetro `per_page` no estaba acotado superiormente, permitiendo consultas que devolvían todos los registros sin límite. Se añadió un máximo de 100 registros por página.
- **Evaluación asíncrona (2 defectos, severidad baja):** el *Evaluation Watchdog* marcaba como fallidas evaluaciones que simplemente estaban pausadas esperando un *worker* Celery disponible. Se ajustó el umbral de tiempo de espera de 5 a 10 minutos.

5.7.4 Retrospectiva

Conseguido: plataforma completamente probada, documentada y lista para la defensa. Cobertura de pruebas unitarias del motor de métricas superior al 85 %.

Estado del producto: DATAQUAL es una plataforma funcional, probada y documentada. Tres suites de pruebas automatizan la verificación de las métricas, el versionado y los *Quality Gates*. La memoria del TFG está redactada y los siete anexos completos. La cobertura del *frontend* sigue siendo insuficiente, lo que queda registrado como deuda técnica en las líneas de trabajo futuro.

Pendiente (trabajo futuro): Swagger/OpenAPI, exportación de informes, WebSockets y pruebas *end-to-end* del *frontend*; documentados en la sección 7.4.

Qué fue bien: la batería de pruebas unitarias detectó tres defectos en el módulo de fingerprinting que habrían pasado desapercibidos, demostrando el valor de las pruebas automatizadas.

Qué fue mal: la cobertura de pruebas del *frontend* es escasa; la priorización de funcionalidad sobre pruebas durante los sprints anteriores dejó la interfaz de usuario sin cobertura automatizada.

Estimación vs. real: 80 h estimadas vs. 90 h reales (+12,5 %).

Lección aprendida: las pruebas deben integrarse en cada sprint, no dejarse para el final. Un sprint de consolidación es útil, pero no puede compensar la deuda técnica de pruebas acumulada.

Acción de mejora (para proyectos futuros): adoptar una política de *definition of done* que exija al menos pruebas unitarias de los módulos nuevos antes de cerrar cada historia de usuario.

5. RESULTADOS: DESARROLLO ITERATIVO POR SPRINTS

Resumen global del proyecto: 364 h estimadas vs. 425 h reales (+16,8 % de desviación total), dentro del rango habitual para proyectos de software de esta envergadura.

Capítulo 6

Resultados y discusión

ESTE capítulo consolida los resultados globales de DATAQUAL una vez completados los seis sprints descritos en el capítulo 5. Presenta los resultados cuantitativos y cualitativos del sistema, verifica el cumplimiento de los objetivos específicos OE1–OE5, identifica las fortalezas y limitaciones de la plataforma, y la compara con las herramientas existentes revisadas en el capítulo 2.

6.1 Resultados alcanzados

DATAQUAL es una plataforma funcional de extremo a extremo que cumple con los cinco objetivos específicos definidos en la sección 3.2.

6.1.1 Resultados cuantitativos

El Cuadro 6.1 resume las principales magnitudes del sistema implementado.

Cuadro 6.1: Resultados cuantitativos de la implementación

Dimensión	Cantidad
Métricas de calidad evaluables (Quality Score)	6
Clases de métricas implementadas (incl. outliers para EDA)	7
Servicios Docker orquestados	8
Rutas / páginas del <i>frontend</i>	20
Componentes React reutilizables	40+
Módulos funcionales de API (<i>blueprints</i> principales)	6
<i>Blueprints</i> Flask totales (incl. sub-rutas REST)	9
Tablas en base de datos	10
Patrones de formato predefinidos (exactitud sintáctica)	13
Tipos de <i>issue</i> (por métrica y subtipo)	10+
Migraciones de base de datos	8+
Suites de pruebas automatizadas	3

6.1.2 Resultados cualitativos

Desde el punto de vista cualitativo, cabe destacar los siguientes logros:

6. RESULTADOS Y DISCUSIÓN

- El **motor de métricas es extensible**: añadir una nueva métrica requiere únicamente crear una clase, registrarla en el METRIC_REGISTRY y documentarla.
- El **sistema de fingerprinting** permite rastrear la evolución de la calidad entre ejecuciones sucesivas, una capacidad que pocas herramientas de código abierto ofrecen.
- Las **Quality Gates** proporcionan una señal clara y automatizable que puede integrarse en *pipelines* de MLOPS.
- El **profiling interactivo** (histogramas, diagramas de caja, matrices de correlación) ofrece un análisis exploratorio completo sin abandonar la plataforma.
- La **protección de PII** está integrada de forma nativa en todas las métricas, no como una capa adicional.

6.1.3 Verificación del cumplimiento de objetivos

El Cuadro 6.2 recoge, para cada objetivo específico, la evidencia de su cumplimiento.

Cuadro 6.2: Verificación del cumplimiento de los objetivos específicos

Obj.	Evidencia	Estado
OE1	Módulo de ingesta con soporte para CSV, XLSX, JSON y Parquet; parseo robusto con matriz de <i>fallback</i> ; almacenamiento en MinIO; extracción automática de esquema; versionado padre-hijo.	Cumplido
OE2	Siete métricas implementadas como <i>plugins</i> ; parametrización completa; sistema de pesos; plantillas reutilizables; ejecución asíncrona con Celery.	Cumplido
OE3	Seis <i>blueprints</i> de API; autenticación JWT; formato de respuesta estandarizado; paginación; validación Marshmallow.	Cumplido
OE4	Más de 20 páginas; cuadros de mando interactivos; EDA con histogramas, diagramas de caja y matrices de correlación; diseño adaptable.	Cumplido
OE5	Sistema de <i>issues</i> con cuatro severidades; <i>fingerprinting</i> SHA-256; comparación con línea base; <i>Quality Gates</i> con tres estados.	Cumplido

6.2 Fortalezas de la plataforma

A partir de los resultados obtenidos, se identifican las siguientes fortalezas de DATA-QUAL:

1. **Arquitectura extensible por plugins.** El patrón BaseMetric + METRIC_REGISTRY permite incorporar nuevas métricas sin modificar el código existente [GHJV94].
2. **Fingerprinting para trazabilidad.** La capacidad de rastrear un mismo *issue* entre ejecuciones (nuevo, recurrente, corregido) es una funcionalidad distintiva respecto a las herramientas de código abierto analizadas en la sección 2.3.
3. **Quality Gates inspirados en SonarQube.** Trasladan un concepto consolidado en la calidad del código [Son24] al dominio de la calidad de datos.

4. **Solución *full-stack* autoalojada.** Todo el *stack* se despliega con un único comando (`docker-compose up`), sin dependencias de servicios en la nube propietarios, garantizando la soberanía de los datos.
5. **Protección nativa de PII.** El enmascaramiento de datos sensibles no es una funcionalidad opcional sino un comportamiento integrado en cada métrica y cada *endpoint*.
6. **Parseo robusto de CSV.** La matriz de *fallback* (codificaciones $\times$ separadores $\times$ motores) gestiona archivos CSV reales que fallan con analizadores estándar.
7. **Métricas específicas para ML.** El equilibrio de clases (entropía de Shannon) y la actualidad no están presentes en la mayoría de las herramientas generalistas de calidad de datos.
8. **Procesamiento asíncrono con resiliencia.** Celery con reintentos, *watchdog* para tareas atascadas y *fallback* síncrono garantizan que las evaluaciones se completen en todos los escenarios.

6.3 Limitaciones

A pesar de los resultados alcanzados, la plataforma presenta las siguientes limitaciones:

1. **Autenticación básica.** Solo se soporta autenticación JWT con usuario y contraseña. No se ha implementado OAuth 2.0, **SSO!** (**SSO!**) ni autenticación multifactor.
2. **Sin exportación de informes.** Los resultados no pueden exportarse a PDF, Excel u otros formatos de informe formal.
3. **Sin documentación interactiva de la API.** La API no dispone de documentación Swagger/OpenAPI generada automáticamente.
4. **Polling en lugar de WebSockets.** El *frontend* consulta periódicamente el estado de las evaluaciones en lugar de recibir notificaciones *push*, lo que introduce latencia y carga innecesaria.
5. **Escalabilidad limitada a un nodo.** Aunque Celery soporta escalado horizontal, la plataforma no se ha validado con múltiples *workers* distribuidos.
6. **Conjuntos de datos en memoria.** Pandas carga el conjunto de datos completo en memoria, por lo que archivos mayores que la RAM disponible no pueden procesarse.
7. **Sin detección de anomalías basada en ML.** Solo se emplean métodos estadísticos clásicos (IQR, Z-score). No se han implementado algoritmos como *Isolation Forest* o DBSCAN.
8. **Cobertura de pruebas limitada.** Las pruebas automatizadas cubren *Quality Gates* y versionado, pero no el flujo completo de evaluación de extremo a extremo.

6.4 Comparación con herramientas existentes

El Cuadro 6.3 retoma la comparación del capítulo 2 enriquecida con perspectiva de implementación: no solo si la funcionalidad existe, sino qué implicó reproducirla o superarla en la práctica.

Cuadro 6.3: Comparación de DATAQUAL con herramientas existentes (perspectiva post-implementación)

Criterio	DATAQUAL	Great Exp.	Deequ	DQOps
UI integrada	Sí	No	No	Sí
Requiere código	No	Sí	Sí	Parcial
<i>Quality Gates</i>	Sí	No	No	Sí
<i>Fingerprinting</i>	Sí	No	No	Parcial
Diff entre ejecuciones	Sí	No	Parcial	Sí
Métricas para ML	Sí	No	Parcial	No
Enmascaramiento PII	Sí	No	No	No
Versionado de datasets	Sí	No	No	No
Autoalojado	Sí	Sí	Sí	Sí
Parseo robusto de archivos	Sí	Parcial	No	No
Madurez	Prototipo	Producción	Producción	Producción

La implementación aportó perspectivas que no eran evidentes en el análisis previo del estado del arte. El **parseo robusto de archivos** resultó ser un diferenciador real: en las pruebas con conjuntos de datos reales, una fracción significativa de los ficheros CSV presentó combinaciones de codificación y delimitador no estándar que ninguna herramienta *code-first* gestiona automáticamente sin configuración explícita. El **enmascaramiento nativo de PII** supuso un coste de implementación considerable —todas las clases de métricas deben respetar la lista de columnas sensibles —pero su valor es especialmente alto en contextos regulados por el RGPD, donde ninguna de las alternativas analizadas lo ofrece de forma integrada.

DATAQUAL no pretende competir con herramientas maduras de producción como Great Expectations [Gre24] en escala o número de métricas predefinidas, pero ocupa un nicho diferenciado al combinar interfaz web completa, métricas específicas para ML, *fingerprinting* determinista, *Quality Gates* y protección nativa de PII en un paquete autoalojado que no requiere escribir código.

Capítulo 7

Conclusiones y trabajo futuro

ESTE capítulo final recoge la revisión punto por punto de los objetivos del proyecto, las conclusiones y valoración global, las lecciones aprendidas, las líneas de trabajo futuro, las competencias adquiridas y una valoración personal del proceso.

7.1 Revisión de objetivos

El presente TFG estableció cinco objetivos específicos (OE1–OE5) y un conjunto de objetivos transversales. A continuación se revisa el grado de cumplimiento de cada uno.

OE1 — Módulo de ingesta y almacenamiento Cumplido. Se implementó soporte para CSV, XLSX, JSON y Parquet con estrategia de *fallback* múltiple (4 codificaciones $\times$ 5 delimitadores), almacenamiento seguro en MinIO, extracción automática de esquema y versionado padre-hijo con indicador `is_latest`.

OE2 — Motor de métricas parametrizables Cumplido. Se implementaron seis métricas de calidad evaluables (completitud, unicidad, exactitud sintáctica, consistencia lógica, equilibrio de clases y actualidad) mediante arquitectura de *plugins* (BaseMetric + METRIC_REGISTRY), con parametrización completa, sistema de pesos y plantillas reutilizables. La clase `OutliersMetric` se implementó como herramienta de diagnóstico disponible en el módulo de perfilado, sin participar en el *Quality Score* (alineado con ISO/IEC 5259).

OE3 — API REST documentada Cumplido. Se implementaron los seis módulos funcionales definidos como *blueprints* Flask principales (`auth`, `projects`, `datasets`, `metrics`, `evaluations`, `admin`), con autenticación JWT, formato de respuesta estandarizado `{success, data, message}`, paginación y validación Marshmallow. La convención REST de rutas anidadas añadió tres *blueprints* técnicos adicionales (`project_metrics`, `project_datasets`, `dashboard`), totalizando nueve *blueprints* registrados. La especificación completa se recoge en el Anexo A.

OE4 — Aplicación web con cuadros de mando interactivos Cumplido. Se desarrollaron más de 20 páginas con cuadros de mando, visualizaciones interactivas (*EDA*: histogramas, boxplots, matrices de correlación) y diseño adaptable.

OE5 — Mecanismos de identificación de problemas Cumplido. Se implementaron el sistema de *issues* con cuatro niveles de severidad, el *fingerprinting* SHA-256 para trazabilidad entre evaluaciones y los *Quality Gates* con tres estados (PASSED/WARNING/FAILED).

Objetivos transversales Cumplidos en su mayoría. Seguridad multicapa (JWT, PBKDF2, rate limiting, cabeceras), escalabilidad asíncrona (Celery + Redis), mantenibilidad (módulos independientes) y portabilidad (Docker Compose) han sido alcanzados. La internacionalización y las pruebas *end-to-end* del *frontend* quedan como trabajo futuro.

Valoración global del proyecto: DATAQUAL es una plataforma funcional de extremo a extremo que supera en funcionalidades específicas para ML (equilibrio de clases, fingerprinting, Quality Gates, PII nativo) a las alternativas open source revisadas. Su madurez es la de un prototipo funcional, no de una herramienta de producción, lo cual está alineado con el alcance de un TFG.

Limitaciones encontradas: ausencia de Swagger/OpenAPI, sin exportación de informes, *polling* en lugar de WebSockets, escalabilidad limitada a un nodo y cobertura de pruebas del *frontend* insuficiente.

7.2 Conclusiones

El presente TFG ha abordado el diseño, la implementación y la validación de DATAQUAL, una plataforma web integral para la evaluación automatizada de la calidad de datos en proyectos de IA, ML y minería de datos. A lo largo de los seis sprints descritos en el capítulo 5, se ha logrado construir un sistema funcional de extremo a extremo que cumple los cinco objetivos específicos establecidos en el capítulo 3, tal como se ha verificado en la sección 6.1.

Las principales conclusiones que se extraen del trabajo son las siguientes:

1. **La calidad de datos requiere herramientas accesibles.** Los enfoques ad hoc basados en *scripts* y cuadernos Jupyter, aunque flexibles, presentan barreras de estandarización, historización y accesibilidad que dificultan su adopción sistemática. DATAQUAL demuestra que es viable ofrecer una interfaz web accesible a perfiles no técnicos sin renunciar a la potencia de una API RESTful para integraciones programáticas.
2. **Los estándares internacionales proporcionan un marco sólido.** La alineación con ISO/IEC 25012 [ISO08] e ISO/IEC 5259 [ISO24] ha proporcionado un marco de referencia riguroso para la definición de las métricas de calidad, evitando definiciones arbitrarias y facilitando la comparabilidad entre proyectos.
3. **La arquitectura de *plugins* facilita la extensibilidad.** El diseño del motor de métricas basado en una clase base abstracta y un registro dinámico ha demostrado ser

una decisión arquitectónica acertada: las seis métricas evaluables se han implementado de forma independiente, y el proceso de incorporación de nuevas métricas está formalizado en cuatro pasos claramente definidos.

4. **El *fingerprinting* habilita la trazabilidad.** La generación de *hashes* deterministas para cada *issue* detectado permite una comparación automática entre ejecuciones que supera las capacidades de la mayoría de las herramientas de código abierto analizadas en el estado del arte.
5. **Las *Quality Gates* aportan claridad a la toma de decisiones.** La traslación del concepto de *Quality Gate* del ámbito de la calidad del código [Son24] al ámbito de la calidad de datos proporciona una señal objetiva y automatizable sobre la aptitud de un conjunto de datos para su uso.
6. **La contenerización garantiza la portabilidad.** El despliegue mediante Docker Compose [Doc24] asegura que la plataforma pueda reproducirse en cualquier entorno sin dependencias externas, respetando la soberanía de los datos.

7.3 Lecciones aprendidas

El desarrollo del proyecto ha proporcionado las siguientes lecciones relevantes desde la perspectiva de la ingeniería del software:

- **El desarrollo iterativo minimiza los riesgos.** La organización en iteraciones con entregas funcionales ha permitido detectar y corregir problemas de diseño de forma temprana, evitando retrabajos costosos en fases avanzadas del proyecto.
- **El diseño de métricas exige conocimiento del dominio.** La implementación de métricas de calidad de datos no es un ejercicio puramente algorítmico: requiere comprender los estándares internacionales, las necesidades reales de los usuarios y los matices de cada dimensión de calidad.
- **La seguridad debe integrarse desde el inicio.** Decisiones como la protección contra inyección en las reglas de consistencia lógica o el enmascaramiento nativo de PII fueron más sencillas de implementar al considerarlas desde las fases iniciales del diseño que si se hubieran añadido como capas posteriores.
- **La gestión de la complejidad del *frontend* requiere disciplina.** Componentes como el módulo de *profiling* de datos crecieron significativamente durante el desarrollo. Una mayor modularización temprana habría facilitado el mantenimiento posterior.

7.4 Líneas de trabajo futuro

A partir de las limitaciones identificadas en la sección 6.3 y de las oportunidades detectadas durante el desarrollo, se proponen las siguientes líneas de trabajo futuro para la evolución de DATAQUAL:

7. CONCLUSIONES Y TRABAJO FUTURO

1. **Documentación Swagger/OpenAPI** para la API RESTful, proporcionando documentación interactiva generada automáticamente.
2. **Exportación de informes** a PDF y Excel para facilitar la comunicación de resultados a *stakeholders* no técnicos.
3. **WebSockets** para notificaciones *push* del progreso de las evaluaciones, sustituyendo el mecanismo actual de *polling*.
4. **OAuth 2.0 y SSO!** para la integración con sistemas empresariales de gestión de identidades.
5. **Evaluaciones programadas** mediante Celery Beat para la monitorización continua de la calidad de datos.
6. **Detección de anomalías basada en ML**, incorporando algoritmos como *Isolation Forest* o *Autoencoders* como métricas adicionales del motor.
7. **Procesamiento por fragmentos** (*chunk-based*) para conjuntos de datos que excedan la memoria RAM disponible.
8. **Integración directa con bases de datos** (PostgreSQL, MySQL, BigQuery) como fuentes de datos, eliminando la necesidad de exportar a archivo.
9. **Colaboración en equipo** con roles y permisos granulares, permitiendo que múltiples usuarios trabajen sobre los mismos proyectos.
10. **Webhooks** para notificar a sistemas externos cuando una evaluación se complete o un *Quality Gate* falle.
11. **Internacionalización** completa de la interfaz de usuario.
12. **Sugerencias de corrección automatizadas** basadas en los *issues* detectados, cerrando el ciclo de diagnóstico con propuestas de acción.

7.5 Aportación del proyecto

7.5.1 Aportación técnica

DATAQUAL aporta al panorama de herramientas de calidad de datos una solución *full-stack* autoalojada que combina en un único producto: interfaz web accesible sin necesidad de programar, métricas específicas para proyectos de ML (equilibrio de clases, actualidad), sistema de *fingerprinting* determinista para trazabilidad de *issues*, *Quality Gates* configurables y protección nativa de PII. Esta combinación no está disponible en ninguna de las herramientas de código abierto analizadas en el estado del arte (sección 2.3).

7.5.2 Aportación académica

El proyecto materializa la convergencia entre la ingeniería del software moderna (arquitecturas de microservicios, patrones de diseño como *plugin* y factoría, CI/CD) y el dominio

emergente de la calidad de datos para IA. Proporciona una implementación de referencia alineada con los estándares ISO/IEC 25012 e ISO/IEC 5259, que puede servir como base para investigación futura sobre métricas de calidad específicas para ML.

7.5.3 Aportación profesional

El proyecto ha permitido al autor desarrollar competencias profesionales en un conjunto amplio de tecnologías de producción: desarrollo *full-stack* (Flask + Next.js), bases de datos relacionales con JSONB, almacenamiento de objetos S3, colas de tareas asíncronas (Celery + Redis) y despliegue contenerizado (Docker Compose), en un contexto de proyecto real con requisitos de seguridad, escalabilidad y mantenibilidad.

7.6 Competencias adquiridas

El desarrollo de este TFG en la intensificación de Sistemas de Información ha permitido adquirir y consolidar las siguientes competencias:

7.6.1 Competencias técnicas

- **Desarrollo *full-stack*:** diseño e implementación de una aplicación web completa con separación clara entre *frontend* (Next.js, React, TypeScript) y *backend* (Python, Flask, SQLAlchemy).
- **Arquitecturas distribuidas y contenerización:** orquestación de ocho servicios Docker con Docker Compose, gestión de dependencias entre servicios, *healthchecks* y volúmenes persistentes.
- **Procesamiento de datos tabulares:** manipulación de datasets en múltiples formatos con Pandas, cálculo de métricas estadísticas y gestión de casos extremos (nulos, tipos mixtos, codificaciones).
- **Calidad de datos e ingeniería de la calidad:** comprensión profunda de los estándares ISO/IEC 25012 e ISO/IEC 5259 y su aplicación práctica en el diseño de métricas.
- **Seguridad en aplicaciones web:** autenticación JWT, *hashing* de contraseñas con PBKDF2, limitación de tasa, cabeceras de seguridad y prevención de inyección de código.
- **Diseño de APIs REST:** especificación y documentación de *endpoints*, gestión de versiones, paginación y validación de entradas.

7.6.2 Competencias metodológicas

- **Gestión ágil con Scrum adaptado:** planificación de sprints, gestión del backlog con GitHub Issues, retrospectivas y adaptación del plan ante imprevistos.
- **Control de versiones avanzado:** estrategia de *feature branches*, mensajes de *commit* descriptivos y trazabilidad entre código e issues.

- **Estimación y planificación:** uso de *story points*, análisis de desviaciones y ajuste de estimaciones a lo largo del proyecto.
- **Documentación técnica:** redacción de una memoria de TFG completa en $\text{\LaTeX}$, especificaciones de API y catálogos de métricas.

7.6.3 Competencias transversales

- **Resolución autónoma de problemas técnicos:** diagnóstico y resolución de problemas de integración entre servicios Docker, incompatibilidades de formato de datos y gestión de errores en sistemas distribuidos.
- **Capacidad de síntesis y abstracción:** diseño de interfaces y abstracciones que permiten extender el sistema (motor de métricas, arquitectura de *plugins*) sin conocer los detalles de las implementaciones concretas.
- **Trabajo autónomo bajo supervisión:** gestión del tiempo y la priorización en un proyecto individual de nueve meses con reuniones de seguimiento periódicas.

7.7 Valoración personal

Este TFG ha supuesto un reto de una dimensión cualitativamente diferente a los proyectos realizados durante el grado. Por primera vez, he tenido que diseñar y construir un sistema completo desde cero, tomando decisiones de arquitectura con consecuencias a largo plazo, integrando tecnologías que no había utilizado antes (Celery, MinIO, Redis en producción) y manteniendo la coherencia de un proyecto de nueve meses en solitario.

Lo que más me ha sorprendido es la complejidad inherente a los detalles: los ficheros CSV del mundo real no se comportan como los de los tutoriales, la configuración de Docker Compose con dependencias entre servicios exige una comprensión profunda de los tiempos de arranque, y el diseño de un *fingerprint* determinista para issues requirió mucho más pensamiento del esperado. Estas dificultades, en retrospectiva, son precisamente las más valiosas porque son las que ocurren en proyectos reales.

Me llevo especialmente la convicción de que una buena arquitectura inicial (el modelo de datos pensado desde el principio, la arquitectura de *plugins* para el motor de métricas) ahorra enormemente el trabajo en sprints posteriores, y que el desarrollo iterativo no es solo una metodología formal sino una forma de pensar que reduce el riesgo real de los proyectos.

Estoy satisfecho con el resultado: DATAQUAL es una plataforma funcional, segura y extensible que cubre un hueco real en el ecosistema de herramientas de calidad de datos para IA. Queda trabajo por hacer (Swagger, WebSockets, pruebas de *frontend*), pero eso es precisamente lo que convierte un TFG en el punto de partida de algo mayor.

ANEXOS

Anexo A

Especificación de la API RESTful

Este anexo documenta los principales *endpoints* de la API RESTful de DATAQUAL, organizados por módulo (*blueprint*). Todos los *endpoints* protegidos requieren un *token* JWT válido en la cabecera `Authorization: Bearer <token>`. Las respuestas siguen el formato estandarizado `{success, data, message}`.

A.1 Módulo de autenticación (/api/auth)

POST /api/auth/register

Registro de un nuevo usuario. Requiere `username`, `email` y `password`.

POST /api/auth/login

Inicio de sesión. Devuelve *access_token* y *refresh_token*.

POST /api/auth/refresh

Refresco del *access token* utilizando el *refresh token*.

POST /api/auth/logout

Cierre de sesión. Revoca el *token* actual.

GET /api/auth/profile

Consulta del perfil del usuario autenticado.

PUT /api/auth/profile

Actualización del perfil del usuario.

A.2 Módulo de proyectos (/api/projects)

GET /api/projects/

Lista de proyectos del usuario autenticado, con conteo de conjuntos de datos y puntuación de la última evaluación. Soporta paginación.

POST /api/projects/

Creación de un nuevo proyecto. Requiere `name` y, opcionalmente, `description`.

GET /api/projects/<id>

Detalle de un proyecto.

PUT /api/projects/<id>

Actualización de un proyecto (nombre, descripción, configuración de métricas).

DELETE /api/projects/<id>

Eliminación de un proyecto y todos sus recursos asociados (conjuntos de datos, evaluaciones, *issues*).

GET /api/projects/<id>/metrics/config

Consulta de la configuración de métricas del proyecto.

POST /api/projects/<id>/metrics/config

Actualización de la configuración de métricas del proyecto.

A.3 Módulo de conjuntos de datos (/api/datasets)

POST /api/projects/<id>/datasets

Carga de un nuevo conjunto de datos (formulario *multipart*). Acepta CSV, XLSX, JSON y Parquet.

GET /api/datasets/<id>

Consulta de metadatos, esquema y estadísticas de un conjunto de datos.

GET /api/datasets/<id>/versions

Historial de versiones de un conjunto de datos.

PUT /api/datasets/<id>/sensitive-columns

Actualización de las columnas marcadas como PII.

GET /api/datasets/<id>/download

Descarga del archivo original.

A.4 Módulo de métricas (/api/metrics)

GET /api/metrics/

Catálogo de métricas disponibles con sus parámetros de configuración.

GET /api/metrics/templates

Lista de plantillas de métricas (del sistema y del usuario).

POST /api/metrics/templates

Creación de una nueva plantilla de métricas.

DELETE /api/metrics/templates/<id>

Eliminación de una plantilla del usuario.

A.5 Módulo de evaluaciones (/api/evaluations)

POST /api/evaluations/

Lanzamiento de una nueva evaluación. Requiere `project_id` y `dataset_id`.

GET /api/evaluations/<id>

Consulta de los resultados completos de una evaluación: métricas, puntuación, *issues* y estado del *Quality Gate*.

GET /api/evaluations/<id>/status

Consulta del estado y progreso de una evaluación en curso.

GET /api/evaluations/<id>/issues

Lista de *issues* detectados en la evaluación, con severidad, descripción, columnas afectadas y muestras de filas.

GET /api/projects/<id>/evaluations

Historial de evaluaciones de un proyecto, con puntuaciones y estados de *Quality Gate*.

A.6 Módulo de administración (/api/admin)

GET /api/admin/health

Comprobación de salud del sistema. Verifica la conectividad con PostgreSQL, MinIO y Redis.

GET /api/admin/performance

Estadísticas de rendimiento por *endpoint*: número de peticiones, tiempo medio, mínimo y máximo.

Anexo B

Catálogo de métricas de calidad

Este anexo recoge el catálogo completo de las métricas implementadas en DATAQUAL. El sistema implementa **siete clases de métricas**: seis son **métricas evaluables** que contribuyen al *Quality Score* (completitud, unicidad, exactitud sintáctica, consistencia lógica, equilibrio de clases y actualidad), y una (outliers) es una **clase de diagnóstico** disponible en el módulo de perfilado de datos pero excluida del *Quality Score*, en línea con ISO/IEC 5259 [ISO24]. Para cada métrica se detalla su fundamento, los parámetros de configuración disponibles y los *issues* que genera.

B.1 Completitud (completeness)

Fundamento. Mide el grado en que los datos esperados están presentes en el conjunto de datos, alineada con la dimensión de completitud de ISO/IEC 25012 [ISO08].

Algoritmo. Ratio de valores no nulos por columna. Si se especifican columnas, la puntuación es la media de sus ratios; en caso contrario, se usa la media global.

Parámetros:

columns

Lista de columnas a evaluar (por defecto, todas).

threshold

Umbral mínimo de completitud (por defecto, 0,95).

weight

Peso de la métrica (por defecto, 1,0).

Issues. Global (completitud por debajo del umbral) y por columna (más del 2 % de valores nulos).

B.2 Unicidad (uniqueness)

Fundamento. Evalúa la ausencia de duplicados a nivel de fila y la variabilidad por columna.

Algoritmo. Unicidad de filas = $filas_nicas / total_filas$. Variabilidad por columna con umbrales adaptativos: 0,95 para columnas de identificador, 0,05 para categóricas y 0,30 para

el resto.

Parámetros:

threshold

Umbral mínimo de unicidad (por defecto, 1,0).

columns

Columnas que deben ser únicas.

weight

Peso de la métrica (por defecto, 1,0).

Issues. Baja variabilidad, filas duplicadas (con muestras enmascaradas si contienen PII) y columna de identificador no única.

B.3 Valores atípicos (outliers)

Fundamento. Detecta valores que se desvían significativamente del resto del conjunto de datos.

Algoritmos.

- **IQR:** un valor es atípico si $x < Q_1 - f \cdot \text{IQR}$ o $x > Q_3 + f \cdot \text{IQR}$, donde f es el factor configurable.
- **Z-score:** un valor es atípico si $|z| = |(x - \mu)/\sigma| > f$.

La puntuación penaliza con factor 3×: $s_{\text{col}} = \max(0, 1 - r_{\text{outliers}} \times 3)$.

Parámetros:

method

Método de detección: "iqr" o "zscore" (por defecto, "iqr").

factor

Factor del método (por defecto, 1,5).

columns

Columnas numéricas a evaluar (por defecto, todas).

weight

Peso de la métrica (por defecto, 1,0).

B.4 Exactitud sintáctica (syntactic_accuracy)

Fundamento. Verifica la conformidad de los valores con patrones de formato predefinidos, alineada con la dimensión de exactitud de ISO/IEC 25012 [ISO08].

Catálogo de tipos. 13 tipos predefinidos: *email*, *url*, teléfono (español e internacional), DNI, fechas (ISO y europea), entero, decimal, UUID, IPv4, código postal y tarjeta de crédito.

Auto-detección. Para columnas sin configuración explícita, se muestrea hasta 100 valores y se asigna el tipo con mayor tasa de coincidencia si supera el 60 %.

Parámetros:

columns

Lista de {column, expected_type, pattern}.

auto_detect_types

Activar auto-detección (por defecto, true).

threshold

Umbral de conformidad (por defecto, 0,95).

weight

Peso de la métrica (por defecto, 1,0).

B.5 Consistencia lógica (logical_consistency)

Fundamento. Valida reglas de negocio entre columnas, expresadas como reglas IF - THEN.

Algoritmo. Cada regla se evalúa mediante `pandas.DataFrame.query()`. La tasa de cumplimiento se calcula como $1 - (violaciones/total_filas)$. Antes de ejecutar cualquier regla, se verifica que no contenga *tokens* prohibidos (import, exec, eval, etc.) para prevenir la inyección de código.

Parámetros:

rules

Lista de reglas: {name, type, expression, condition, assertion}.

weight

Peso de la métrica (por defecto, 1,0).

B.6 Equilibrio de clases (class_balance)

Fundamento. Métrica específica para ML supervisado. Utiliza la entropía de Shannon [Sha48] para cuantificar el equilibrio de la variable objetivo:

$$H = - \sum_i p_i \cdot \log_2(p_i), \quad H_{\text{máx}} = \log_2(n_{\text{clases}}), \quad \text{Balance} = \frac{H}{H_{\text{máx}}} \times 100$$

Auto-detección. Se analizan automáticamente las columnas de tipo object o category con hasta 50 valores únicos, y las columnas enteras con hasta 20 valores únicos.

Parámetros:

columns

Columnas objetivo.

auto_detect

Activar auto-detección (por defecto, true).

imbalance_threshold_high

Umbral de clase dominante (por defecto, 0,90).

imbalance_threshold_low

Umbral de clase minoritaria (por defecto, 0,05).

weight

Peso de la métrica (por defecto, 1,0).

Issues. Clase dominante ($\geq 90\%$) y clase minoritaria ($\leq 5\%$).

B.7 Actualidad (currentness)

Fundamento. Mide la frescura de los datos calculando la antigüedad del valor de fecha más reciente de cada columna temporal, alineada con la dimensión de actualidad (*currentness*) de ISO/IEC 25012 [ISO08].

Algoritmo de frescura. La degradación es lineal entre el umbral de obsolescencia y el doble de dicho umbral:

$$frescura = \begin{cases} 1,0 & \text{si } das \leq umbral \\ \text{máx}(0, 1 - \frac{das - umbral}{umbral}) & \text{en caso contrario} \end{cases}$$

Auto-detección. Columnas con dtype=datetime64 se incluyen directamente; para columnas de texto, se parsea una muestra de 50 valores y se incluyen si al menos el 50 % se reconoce como fecha.

Parámetros:

columns

Columnas temporales a evaluar.

auto_detect

Activar auto-detección (por defecto, true).

staleness_threshold_days

Umbral de obsolescencia en días (por defecto, 30).

weight

Peso de la métrica (por defecto, 1,0).

Anexo C

Backlog completo del proyecto

Este anexo recoge el backlog completo del proyecto con todas las historias de usuario e ítems técnicos, organizados por sprint. Los ítems marcados con asterisco (*) no estaban en el backlog inicial y se añadieron durante el desarrollo.

Sprint 1 (octubre–noviembre 2025)

Cuadro C.1: Backlog Sprint 1

ID	Ítem	SP	Estado
HU-01	Registro con email y contraseña	3	Cerrado
HU-02	Login y JWT (access + refresh token)	2	Cerrado
T-01	Setup Docker Compose (8 servicios + healthchecks)	3	Cerrado
T-02	Diseño ERD y modelo de datos inicial	3	Cerrado
T-03	Migraciones iniciales con Flask-Migrate	2	Cerrado

Sprint 2 (diciembre 2025–enero 2026)

Cuadro C.2: Backlog Sprint 2

ID	Ítem	SP	Estado
HU-03	CRUD de proyectos	3	Cerrado
HU-05	Carga de archivos (CSV, XLSX, JSON, Parquet)	5	Cerrado
HU-06	Versionado de datasets (parent-child)	3	Cerrado
T-04	Arquitectura backend en capas (blueprints + servicios)	4	Cerrado
T-05	Pruebas manuales de la API REST	3	Cerrado
T-06*	Parseo robusto CSV con matriz de fallback (4×5×2)	2	Cerrado

Sprint 3 (febrero 2026)

Sprint 4 (marzo 2026)

Sprint 5 (abril 2026)

Sprint 6 (mayo–junio 2026)

Cuadro C.3: Backlog Sprint 3

ID	Ítem	SP	Estado
HU-07	Configuración de métricas con parámetros y pesos	5	Cerrado
HU-08	Plantillas de métricas del sistema	3	Cerrado
HU-09	Lanzamiento de evaluación (Celery) + progreso	5	Cerrado
HU-10	Listado de issues con severidad y columna afectada	3	Cerrado

Cuadro C.4: Backlog Sprint 4

ID	Ítem	SP	Estado
HU-04	Cuadro de mando principal con resumen de calidad	5	Cerrado
HU-09	Progreso de evaluación en tiempo real (polling)	3	Cerrado
HU-13	EDA: histogramas, boxplots, correlaciones	5	Cerrado
HU-14	Marcado de columnas PII en la UI	1	Cerrado

Cuadro C.5: Backlog Sprint 5

ID	Ítem	SP	Estado
HU-11	Quality Gates con tres estados (PASSED/WARNING/FAILED)	5	Cerrado
HU-12	Fingerprinting SHA-256 para trazabilidad de issues	5	Cerrado
T-07	Seguridad: rate limiting, cabeceras, PBKDF2	0	Cerrado
T-08*	HybridEvaluationService (fallback síncrono)	2	Cerrado
T-09*	Evaluation Watchdog (detección de evaluaciones atascadas)	1	Cerrado

Cuadro C.6: Backlog Sprint 6

ID	Ítem	Estado
HU-15	Panel de administración (/api/admin/)	Cerrado
T-10	Suite de pruebas unitarias de métricas (pytest)	Cerrado
T-11	Suite de pruebas de integración: versionado datasets	Cerrado
T-12	Suite de pruebas de Quality Gates	Cerrado
T-13	Corrección de defectos detectados en pruebas (11 issues)	Cerrado
T-14	Redacción memoria TFG + Anexo A y B	Cerrado

Anexo D

Plan de riesgos detallado

Este anexo amplía el plan de riesgos resumido en la sección 4.10 con el registro de ocurrencia real de cada riesgo y las acciones correctivas aplicadas durante el proyecto.

Resumen de ocurrencia: de los ocho riesgos identificados, tres se materializaron (R-02, R-03, R-08) y uno tuvo un efecto positivo (R-01). Los cuatro riesgos restantes no ocurrieron, lo que valida las acciones preventivas adoptadas.

Cuadro D.1: Registro de riesgos con ocurrencia y acciones aplicadas

ID	Riesgo	P	I	Preventiva	Correctiva	Ocurrió
R-01	Complejidad del motor de métricas	M	A	Sprint completo reservado para S3	Alcance de exactitud sintáctica ampliado a 13 tipos (mejora)	Parcialmente (positivo)
R-02	Problemas integración Docker	M	A	Stack completo desde S1	Ajuste de health-checks y tiempos de reconexión en S5	Sí (S5, Celery+Redis)
R-03	Desvío temporal	A	M	Margen del 20 % en estimaciones	Repriorización del backlog; HU-13 reducida en alcance	Sí (S2 y S5)
R-04	Pérdida de datos/código	B	A	Push diario a GitHub	N/A	No
R-05	Cambios de requisitos por tutores	M	M	Revisiones quincenales	Añadidos ítems al backlog (T-08, T-09) en S5	Sí (menor)
R-06	Rendimiento con ficheros grandes	M	M	Límite de 100 MB + chunking	N/A (dentro del límite en todos los tests)	No
R-07	Fallos auth JWT en producción	B	A	Pruebas de integración del flujo auth	N/A	No
R-08	Incompatibilidad de versiones	M	M	Versiones fijas en require-ments.txt	Actualización de SQLAlchemy-chemistry 1.x → 2.0 en S2	Sí (menor, S2)

P = Probabilidad (A=Alta, M=Media, B=Baja) I = Impacto (A=Alto, M=Medio, B=Bajo)

Anexo E

Estimación de costes y viabilidad económica

Este anexo amplía el análisis de costes de la sección 4.9 con un desglose detallado por sprint y un análisis de viabilidad económica.

Desglose de horas reales por sprint

Cuadro E.1: Horas reales de desarrollo por sprint y categoría

Sprint	Backend	Frontend	Infraestr.	Pruebas	Total
S1 — Análisis y arquitectura	20	0	30	0	60
S2 — Backend core	65	0	10	10	85
S3 — Motor de métricas	55	0	5	10	70
S4 — Frontend	10	50	0	5	65
S5 — Features avanzadas	40	10	5	0	55
S6 — Pruebas y documentación	10	0	0	30	40
S6 — Documentación (memoria)	0	0	0	0	50
Total	200	60	50	55	425

Coste total del proyecto

Análisis de viabilidad

Para evaluar la viabilidad de convertir DATAQUAL en un producto SaaS, se estiman los costes de infraestructura en un entorno AWS:

Con un coste de infraestructura de aproximadamente 90 €/mes, un modelo de suscripción de 15 €/usuario/mes requeriría solo 6 usuarios de pago para cubrir los costes operativos, lo que indica una **viabilidad económica favorable** para el modelo SaaS.

Cuadro E.2: Desglose completo de costes del proyecto

Categoría	Concepto	Uds.	€/ud.	Total
Mano de obra	Desarrollador full-stack junior (425h real)	425 h	25 €/h	10 625 €
Mano de obra	Supervisión tutores (2 tutores × 10h)	20 h	60 €/h	1 200 €
Software	Python, Flask, Next.js, PostgreSQL, Redis, MinIO	Licencias open source	0 €	0 €
Software	GitHub (plan Free)	1	0 €	0 €
Software	Docker Desktop (personal, gratuito)	1	0 €	0 €
Hardware	Equipo personal (amortización 3 años, 9 meses uso)	9 meses	28 €/mes	252 €
Hardware	Electricidad (estimada 50W × 8h/día × 270 días)	108 kWh	0,20 €/kWh	22 €
TOTAL				12 099 €

Cuadro E.3: Estimación de costes de producción en AWS (por mes)

Servicio AWS	Equivalente	€/mes
Amazon RDS (db.t3.small)	PostgreSQL	30 €
Amazon S3 (10 GB)	MinIO	0,25 €
Amazon ElastiCache (cache.t3.micro)	Redis	15 €
Amazon ECS (2 tareas)	Backend + Celery	40 €
Amazon CloudFront + S3	Frontend estático	5 €
Total infraestructura cloud/mes		~90 €

Anexo F

Manual de usuario

Este anexo proporciona una guía paso a paso para las operaciones principales de DATA-QUAL. Se asume que el sistema está desplegado y accesible en `http://localhost:3000`.

Requisitos previos

1. Docker Desktop instalado y en ejecución.
2. Repositorio clonado: `git clone <url-repositorio>`.
3. Fichero `backend/.env` configurado con las variables de entorno (plantilla en `backend/.env.example`).

Arranque del sistema

```
cd TFG-CALIDAD-DATOS-IA
docker-compose up --build    # Primera vez (compila imágenes)
docker-compose up            # Arranques posteriores
```

El sistema tarda aproximadamente 30–60 segundos en estar completamente operativo. Acceder a `http://localhost:3000`.

Flujo principal de uso

1. Registro e inicio de sesión

1. Acceder a `/auth/register` y completar el formulario con nombre, email y contraseña.
2. Iniciar sesión en `/auth/login`. El token JWT se gestiona automáticamente.

2. Crear un proyecto

1. En el cuadro de mando, pulsar el botón **Nuevo proyecto**.
2. Introducir nombre y descripción.
3. El proyecto aparece en la lista con estado *Sin evaluaciones*.

3. Cargar un dataset

1. Acceder al proyecto y pulsar **Cargar dataset**.

2. Seleccionar un archivo CSV, XLSX, JSON o Parquet (máx. 100 MB).
3. El sistema extrae automáticamente el esquema y estadísticas descriptivas.
4. Opcional: marcar columnas sensibles (PII) en la pestaña *Columns*.

4. Configurar métricas

1. En la página del proyecto, pestaña *Métricas*.
2. Seleccionar las métricas deseadas de la lista o usar una plantilla del sistema.
3. Ajustar parámetros: umbrales, pesos y columnas objetivo.
4. Pulsar **Guardar configuración**.

5. Ejecutar una evaluación

1. En la página del proyecto, pulsar **Nueva evaluación**.
2. Seleccionar el dataset a evaluar.
3. La evaluación se lanza de forma asíncrona. La barra de progreso muestra el avance (0–100 %).
4. Al completarse, se muestra el *Quality Score* y el estado del *Quality Gate*.

6. Consultar resultados

1. Pulsar sobre una evaluación completada para ver el detalle.
2. El indicador circular muestra el *Quality Score* (0–100).
3. Las pestañas muestran los resultados por métrica y la lista de *issues* con severidad, columna afectada y descripción.
4. Los *badges* indican si los issues son nuevos, recurrentes o corregidos respecto a la evaluación anterior.

7. Explorar datos (EDA)

1. En la página del dataset, pestaña *Profiling*.
2. Seleccionar columnas para visualizar histogramas y diagramas de caja.
3. La pestaña *Correlaciones* muestra la matriz de correlación de Pearson (numéricas) y Spearman (mixtas).
4. La pestaña *Outliers* permite ajustar el factor IQR interactivamente.

8. Configurar un Quality Gate

1. En la página del proyecto, pestaña *Quality Gate*.
2. Establecer: puntuación mínima (defecto: 70), máximo de issues críticos (defecto: 0) y máximo de issues nuevos (defecto: 10).

3. Los estados son: PASSED (verde), WARNING (amarillo), FAILED (rojo).

Monitorización del sistema

- **Flower** (monitor de Celery): <http://localhost:5555>
- **MinIO Console**: <http://localhost:9001> (usuario/contraseña en .env)

Apagado del sistema

```
docker-compose down          # Apagar (conserva datos en volúmenes)
docker-compose down -v       # Apagar y eliminar volúmenes (borra datos)
```

Anexo G

Diagramas UML del sistema

Este anexo recoge las descripciones estructuradas de los principales diagramas **UML** del sistema. Los diagramas visuales se generarán con PlantUML o draw.io y se incluirán como figuras en la versión final de la memoria.

Diagrama de casos de uso

- **Actor: Usuario autenticado**
 - UC-01: Gestionar proyectos (crear, editar, eliminar, listar)
 - UC-02: Gestionar datasets (cargar, versionar, marcar PII)
 - UC-03: Configurar métricas (seleccionar, parametrizar, plantillas)
 - UC-04: Ejecutar evaluación (lanzar, monitorizar progreso, ver resultados)
 - UC-05: Gestionar Quality Gate (configurar umbrales, consultar estado)
 - UC-06: Explorar datos (EDA: histogramas, boxplots, correlaciones)
- **Actor: Administrador** (hereda de Usuario)
 - UC-07: Gestionar usuarios
 - UC-08: Ver métricas del sistema (rendimiento, salud)
- **Actor: Sistema Celery** (interno)
 - UC-09: Ejecutar tarea de evaluación asíncrona
 - UC-10: Detectar evaluaciones atascadas (Watchdog)

Diagrama entidad–relación (ERD)

Entidades y relaciones principales del esquema PostgreSQL:

- users (1) — (N) projects
- projects (1) — (N) datasets
- projects (1) — (N) evaluations
- projects (1) — (1) quality_gates
- datasets (1) — (N) datasets [versionado: parent_dataset_id]

- evaluations (1) — (N) issues
- evaluations (1) — (1) analysis_runs
- analysis_runs (1) — (N) data_quality_issues
- metric_templates (N) — referenciados por projects.metrics_config (JSONB)

Diagrama de secuencia: Ejecución de evaluación

Flujo principal de la ejecución de una evaluación (camino feliz):

1. **Frontend** → POST /api/evaluations/run [dataset_id, project_id]
2. **Backend** → Crea registro Evaluation (estado: PENDING)
3. **Backend** → Encola tarea Celery run_evaluation
4. **Backend** → Devuelve {evaluation_id} al Frontend
5. **Frontend** → Polling GET /api/evaluations/{id}/status (cada 2s)
6. **Celery Worker** → Descarga dataset de MinIO
7. **Celery Worker** → Invoca EvaluationService
8. **EvaluationService** → Ejecuta cada métrica configurada
9. **EvaluationService** → Calcula Quality Score
10. **EvaluationService** → Genera fingerprints y compara con baseline
11. **EvaluationService** → Evalúa Quality Gate
12. **Celery Worker** → Actualiza Evaluation y AnalysisRun en PostgreSQL (estado: COMPLETED)
13. **Frontend** → Detecta estado COMPLETED → Redirige a página de resultados

Diagrama de clases: Motor de métricas

Jerarquía de clases del motor de métricas (backend/services/metrics/):

- **BaseMetric** (abstracta)
 - Atributos: metric_id: str, name: str, threshold: float, weight: float
 - Métodos: evaluate(df, params) → MetricResult (abstracto), _get_severity(), _mask_pii(), _infer_column_type()
- **CompletenessMetric** hereda BaseMetric
- **UniquenessMetric** hereda BaseMetric
- **OutliersMetric** hereda BaseMetric
- **SyntacticAccuracyMetric** hereda BaseMetric

- **LogicalConsistencyMetric** hereda BaseMetric
- **ClassBalanceMetric** hereda BaseMetric
- **CurrentnessMetric** hereda BaseMetric
- **MetricResult** (dataclass): metric_id, score, details, issues: List[Issue]
- **METRIC_REGISTRY**: Dict[str, Type[BaseMetric]]

Referencias

- [AGNP18] Ziawasch Abedjan, Lukasz Golab, Felix Naumann, y Thorsten Papenbrock. *Data Profiling*. Synthesis Lectures on Data Management. Morgan & Claypool Publishers, 2018.
- [Bay24] Mike Bayer. SQLAlchemy: The Database Toolkit for Python. <https://www.sqlalchemy.org/>, 2024.
- [BG18] Joy Buolamwini y Timnit Gebru. Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification. En *Proceedings of the 1st Conference on Fairness, Accountability and Transparency*, volume 81 of *Proceedings of Machine Learning Research*, páginas 77–91. PMLR, 2018.
- [Cel24] Celery Project. Celery: Distributed Task Queue. <https://docs.celeryq.dev/>, 2024.
- [Cha24] Chart.js Contributors. Chart.js: Simple yet flexible JavaScript charting for designers & developers. <https://www.chartjs.org/>, 2024.
- [Das18] Jeffrey Dastin. Amazon scraps secret AI recruiting tool that showed bias against women. *Reuters*, October 2018.
- [Doc24] Docker, Inc. Docker: Accelerated Container Application Development. <https://www.docker.com/>, 2024.
- [DQO24] DQOps. DQOps — Data Quality Operations Center. <https://dqops.com/>, 2024. Consultado: marzo 2026.
- [Fie00] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. Capítulo 5: Representational State Transfer (REST).
- [Gar17] Gartner. How to Improve Your Data Quality. *Gartner Research*, 2017. Estimación: 12,9 millones de dólares anuales por organización.

- [GHJV94] Erich Gamma, Richard Helm, Ralph Johnson, y John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [Gre24] Great Expectations. Great Expectations: Always know what to expect from your data. <https://greatexpectations.io/>, 2024. Consultado: marzo 2026.
- [ISO08] ISO/IEC. ISO/IEC 25012:2008 — Software engineering — Software product Quality Requirements and Evaluation (SQuaRE) — Data quality model. Standard, International Organization for Standardization, 2008.
- [ISO15] ISO/IEC. ISO/IEC 25024:2015 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of data quality. Standard, International Organization for Standardization, 2015.
- [ISO24] ISO/IEC. ISO/IEC 5259 — Data quality for analytics and machine learning (ML). Standard, International Organization for Standardization, 2024. Partes 1 y 2 publicadas en 2024; partes 3 y 4 en proceso de publicación.
- [Kag22] Kaggle. State of Data Science and Machine Learning 2022. *Kaggle Survey*, 2022. 60–80 % del tiempo dedicado a preparación de datos.
- [KKH23] Dominik Kreuzberger, Niklas Kühn, y Sebastian Hirschl. Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, 11:31866–31879, 2023.
- [Met24] Meta Platforms, Inc. React: A JavaScript library for building user interfaces. <https://react.dev/>, 2024.
- [Mic24] Microsoft. TypeScript: JavaScript With Syntax For Types. <https://www.typescriptlang.org/>, 2024.
- [Min24] MinIO, Inc. MinIO: High Performance Object Storage. <https://min.io/>, 2024.
- [MUI24] MUI. Material UI: React components for faster and easier web development. <https://mui.com/>, 2024.
- [Ng21] Andrew Ng. A Chat with Andrew on MLOps: From Model-centric to Data-centric AI. <https://www.deeplearning.ai/the-batch/andrew-ng-data-centric-ai/>, 2021. DeepLearning.AI.
- [Par16] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (Reglamento General de Protección de Datos). Diario Oficial de la Unión Europea, L 119, pp. 1–88, 2016.

- [Par24] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). Diario Oficial de la Unión Europea, 2024.
- [Red98] Thomas C. Redman. The Impact of Poor Data Quality on the Typical Enterprise. *Communications of the ACM*, 41(2):79–82, 1998.
- [Red01] Thomas C. Redman. *Data Quality: The Field Guide*. Digital Press, 2001.
- [Red24] Redis Ltd. Redis: In-memory data structure store. <https://redis.io/>, 2024.
- [Ron24] Armin Ronacher. Flask: A Python Microframework. <https://flask.palletsprojects.com/>, 2024.
- [Sha48] Claude E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [SKH⁺21] Nithya Sambasivan, Shivani Kapania, Hannah Highfill, Diana Akrong, Praveen Paritosh, y Lora M. Aroyo. “Everyone wants to do the model work, not the data work”: Data Cascades in High-Stakes AI. En *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*. ACM, 2021.
- [SLS⁺18] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, y Andreas Grafberger. Automating Large-Scale Data Quality Verification. En *Proceedings of the VLDB Endowment*, volume 11, páginas 1781–1794, 2018. Apache Deequ.
- [Son24] SonarSource. SonarQube: Code Quality and Security. <https://www.sonarsource.com/products/sonarqube/>, 2024.
- [SS20] Ken Schwaber y Jeff Sutherland. The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game. <https://scrumguides.org/scrum-guide.html>, 2020. Versión noviembre 2020.
- [The24] The PostgreSQL Global Development Group. PostgreSQL: The World’s Most Advanced Open Source Relational Database. <https://www.postgresql.org/>, 2024.
- [Tuk77] John W. Tukey. *Exploratory Data Analysis*. Addison-Wesley, 1977.
- [Ver24] Vercel. Next.js: The React Framework for the Web. <https://nextjs.org/>, 2024.
- [WS96] Richard Y. Wang y Diane M. Strong. Beyond Accuracy: What Data Quality Means to Data Consumers. *Journal of Management Information Systems*, 12(4):5–33, 1996.

Este documento fue editado y tipografiado con L^AT_EX
empleando la clase **gita-tfg** que se puede encontrar en:

<https://github.com/anarubioruiz/gita-tfg>

La clase **gita-tfg** está basada en la clase **esi-tfg**:

<https://github.com/UCLM-ESI/esi-tfg>

[respeta esta atribución de autoría]